

MODULE 41

Containerization with Docker

Package Java applications and their dependencies into lightweight, portable containers -- build, run, and manage them with Docker, from Dockerfile to registry.









MODULE 41 OVERVIEW

Build, run, and manage containerized applications using Docker, and prepare the CRM backend for cloud-native deployment.

This module packages the CRM backend into a small, reproducible, non-root Docker image using a multi-stage build -- the same containerization discipline every production Java service needs before Kubernetes.

DURATION & LAB



1 Hour Theory

Containers, Docker architecture, images, Dockerfiles, multi-stage builds, networking, and registries.



2 Hours Lab

lab41-crm: multi-stage Dockerfile, non-root user, health check, and a container runbook for crm-api.



Scenario

NorthStar CRM (Spring Boot) must run identically from laptop to production -- no root, no baked-in secrets.

MODULE ARC



Understand Containers

Why containers matter, VMs vs containers, and Docker's client-server architecture.



Master Docker

Build, optimize, and multi-stage Dockerfiles; run, network, and configure containers.



Build the Lab

Containerize crm-api with a non-root, health-checked, registry-ready image.

KEY TAKEAWAY Total time: 3 hours (1 hour theory + 2 hours hands-on lab). By the end, you will build, run, and publish a small, secure, production-ready container image.

WHY CONTAINERS MATTER

Containers package applications and their dependencies into a consistent, lightweight, portable unit that runs the same way everywhere.

THE REAL-WORLD PROBLEM

- Different OS distributions between dev, test, and prod.
- Inconsistent dependencies and configuration drift.
- The classic "works on my machine" problem.

Result: delays, failures, and high operational cost.

WHY IT MATTERS FOR ENTERPRISES

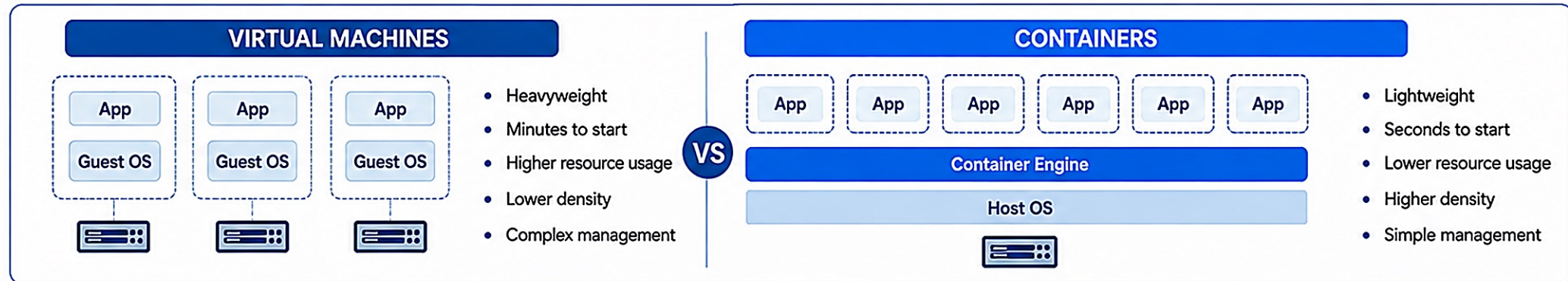
- **Consistency Across Environments** — eliminate "works on my machine" problems.
- **Faster Delivery** — lightweight images start in seconds, speeding up CI/CD.
- **Resource Efficiency** — share the host OS kernel; use fewer resources than VMs.
- **Better Isolation & Security** — process isolation and controlled access reduce risk.
- **Scalability & Flexibility** — scale up or down quickly for microservices architectures.
- **Cloud & Hybrid Ready** — run on laptops, servers, VMs, or any cloud (AWS, Azure, GCP).

CONTAINERS PROVIDE THE SOLUTION



KEY TAKEAWAY Containers bring consistency, speed, and efficiency to application delivery, enabling teams to innovate faster and operate with confidence.

WHY CONTAINERS MATTER



Deliver applications faster, run reliably anywhere, and adapt quickly to change.



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to containerize applications using Docker confidently and prepare them for cloud-native deployment.

- **1. Explain Containerization** — understand what containerization is and why containers matter for modern applications.
- **2. Compare Technologies** — differentiate between virtual machines and containers and identify the key benefits of containers.
- **3. Understand Docker** — describe Docker's architecture and its core components: Engine, Images, Containers, Volumes, Networks, Registries.
- **4. Work with Docker Images** — create, inspect, and optimize Docker images and understand image layers and caching.
- **5. Write Effective Dockerfiles** — write Dockerfiles using best practices and leverage multi-stage builds to create lean, secure images.
- **6. Manage Container Lifecycle** — build, run, stop, and remove containers and understand their lifecycle and states.
- **7. Configure Networking & Environment** — configure container networking and use environment variables to manage configuration.
- **8. Use Registries** — publish and pull images from Docker registries and understand tagging and versioning strategies.
- **9. Apply Best Practices** — apply security, optimization, and operational best practices for building and running containers.
- **10. Prepare for Cloud-Native** — understand how containers enable portability, scalability, and integration with Kubernetes and cloud platforms.

KEY TAKEAWAY Containers bring consistency, portability, and speed to application delivery. Mastering Docker is your first step toward cloud-native success.

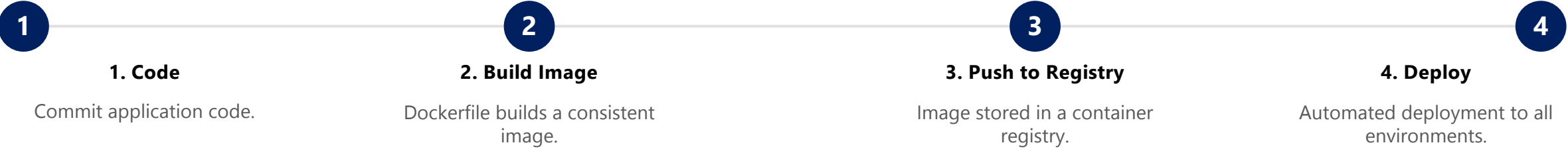
ENTERPRISE SCENARIO: CONSISTENT APPLICATION DEPLOYMENT

Containers ensure our application runs the same way in every environment -- from a developer laptop to production in the cloud.

NorthStar CRM is the same Java Spring Boot crm-api built in Labs 39-40. Operations must deploy it across Dev, Test, Staging, and Prod with identical behavior, fewer issues, and faster delivery.

Environment	Before Containers	After Containers
Developer Laptop	Windows 11, JDK 17, local DB -- "works on my machine."	Docker Desktop, same image, same config -- predictable.
Test Server	Ubuntu, JDK 11, different libraries -- fails in test.	Docker Engine, same image, same config -- consistent.
Staging	Different DB version, manual configs, unexpected bugs.	Docker Engine/K8s, same image, same config -- reliable.
Production	High downtime risk, slow rollback, high operational cost.	Docker/Kubernetes, same image -- high availability.

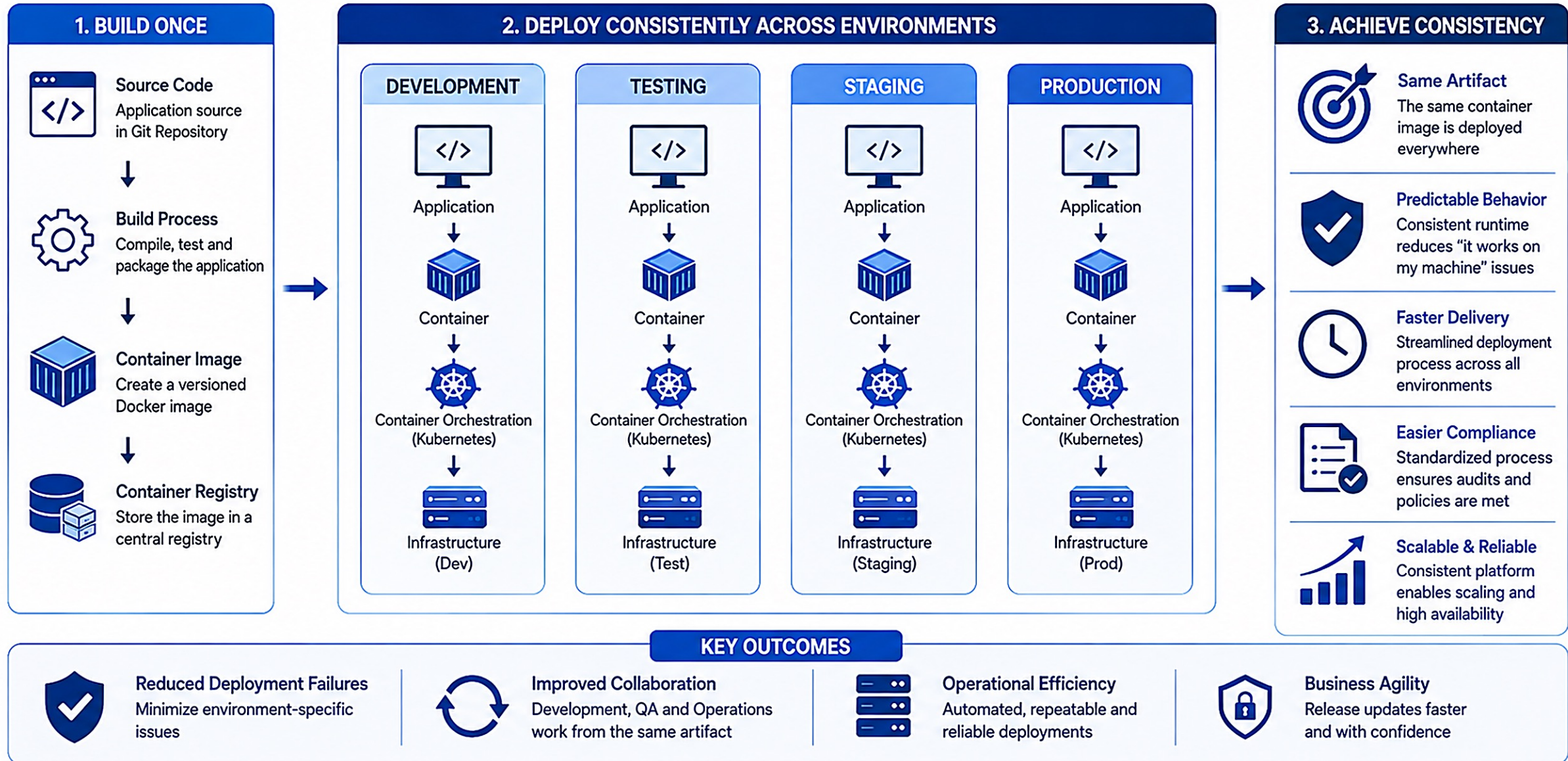
THE CONTAINERIZED DEPLOYMENT FLOW



KEY TAKEAWAY Containers package the application and its dependencies to deliver consistency, portability, and speed -- enabling teams to innovate and operate with confidence.

Enterprise Scenario: Consistent Application Deployment

One Application. Consistent Deployment Across All Environments.



WHAT IS CONTAINERIZATION?

Containerization is an OS-level virtualization approach that packages an application and all its dependencies into a single unit called a container.

WHY CONTAINERIZATION?

- **Consistency** — runs the same way everywhere -- dev, test, staging, prod.
- **Portability** — move containers across laptops, data centers, and clouds.
- **Efficiency** — lightweight and fast startup; uses fewer resources.
- **Isolation & Security** — isolated from other containers and the host system.
- **Scalability** — easily scale up or down to meet demand.

TRADITIONAL VS. CONTAINERIZED DEPLOYMENT

Traditional (Challenges)	Containerized (Solution)
App depends on the underlying OS and its configuration.	Container packages app and dependencies together.
Different environments lead to inconsistencies.	Same image runs consistently everywhere.
Hard to migrate, scale, and maintain.	Easy to move, scale, and automate.

HOW CONTAINERIZATION WORKS



WHAT'S INSIDE A CONTAINER?

- Application code + libraries & dependencies + runtime environment + configuration files + system tools -- a portable, consistent, isolated unit.

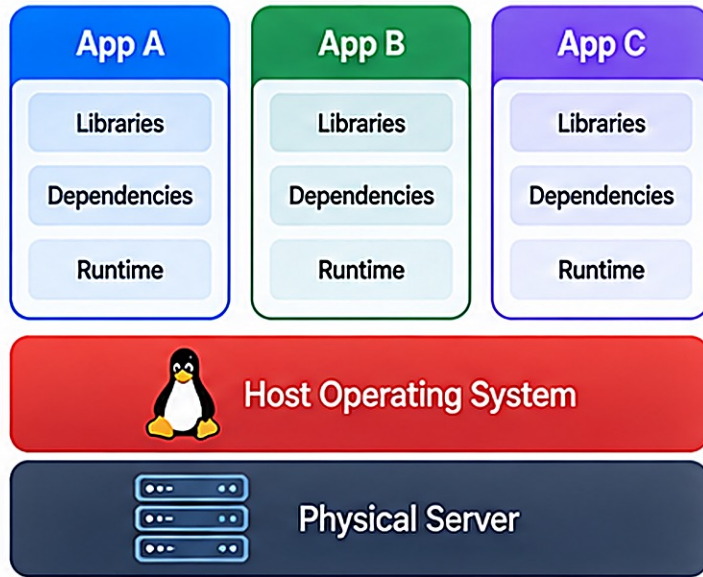
KEY TAKEAWAY Containerization packages your application and everything it needs to run into a lightweight, portable unit, ensuring consistency, efficiency, and speed across all environments.

What is Containerization?



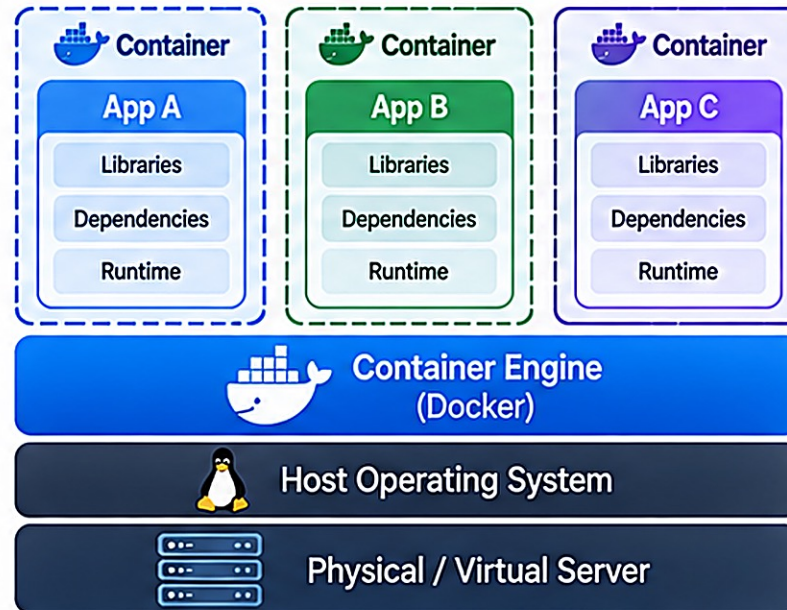
Containerization is a lightweight virtualization technology that packages an application and all its dependencies into a single, portable unit called a **container**.

Traditional Deployment



Applications share the host OS.
Dependency conflicts and environment differences.

With Containerization



Each application runs in its own isolated container
with everything it needs.

Result



Portable, Consistent and Isolated
Application Package



Key Benefits



Consistent
Environments



Portable Across
Environments



Faster Deployment



Easy Scaling



Better Isolation
and Security



VIRTUAL MACHINES VS CONTAINERS

Both provide isolation and portability, but containers are lighter, faster, and more efficient because they share the host OS kernel.

Feature	Virtual Machines	Containers
Architecture	Each VM includes a full OS + app + libraries.	Containers share the host OS kernel.
Startup Time	Minutes.	Seconds.
Resource Usage	High (more CPU, memory, storage).	Low (lightweight).
Density	Lower (few VMs per server).	Higher (many containers per server).
Portability	Good, but larger images.	Excellent, small images.
Management	Complex (OS patching, updates, reboots).	Simpler (immutable images, fast deploys).
Cost	Higher infrastructure cost.	Lower infrastructure cost.
Isolation	Strong (hardware-level, via hypervisor).	Process-level (via OS kernel features).

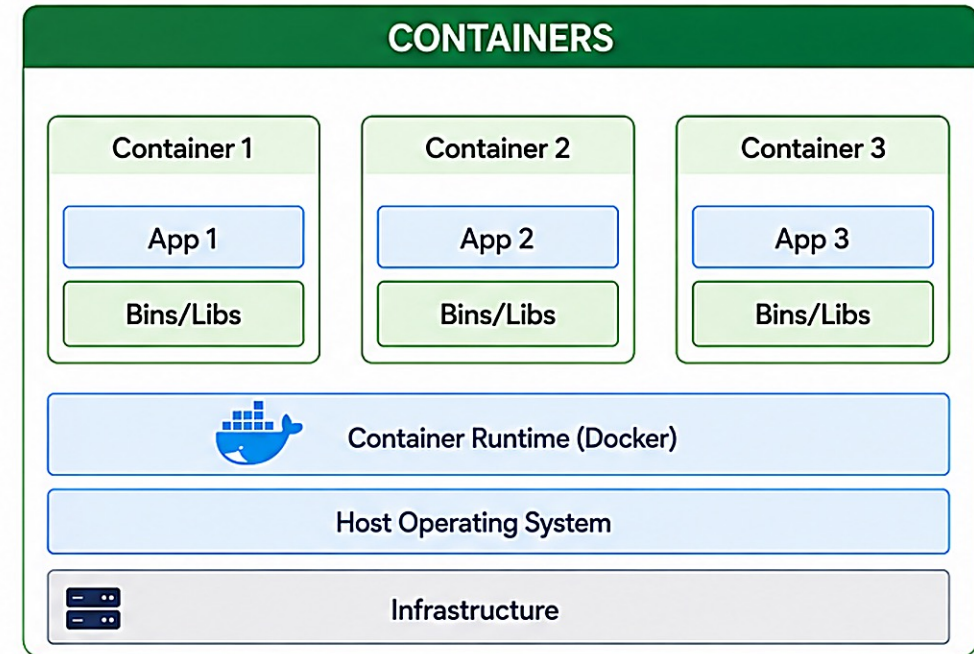
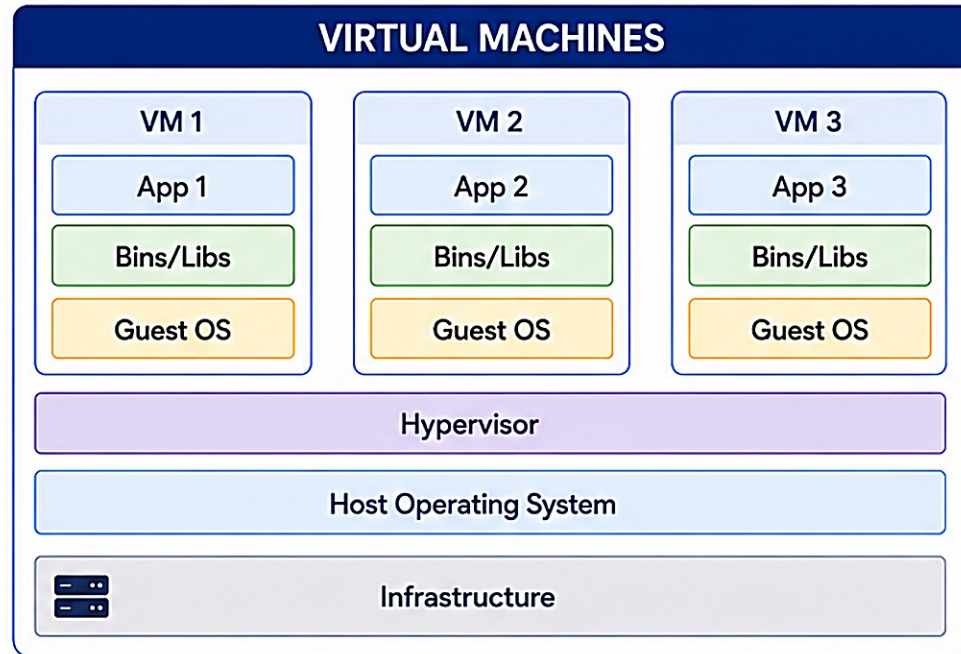
WHEN TO USE WHAT

- **Use VMs when:** you need different OSes, strong isolation, legacy apps, or full control over the OS.
- **Use containers when:** you need fast lightweight deployments, microservices, and CI/CD with rapid scaling.

KEY TAKEAWAY Use Virtual Machines when you need complete OS isolation. Use Containers when you need lightweight, fast, and scalable application deployment.



Virtual Machines vs Containers



ASPECT		VIRTUAL MACHINES	CONTAINERS
	Virtualization Level	Hardware-level virtualization	OS-level virtualization
	Guest OS	Each VM includes a full Guest OS	Share the host OS kernel
	Resource Usage	Higher (more overhead)	Lower (lightweight)
	Startup Time	Slower (minutes)	Faster (seconds)
	Density	Lower (fewer per host)	Higher (more per host)
	Portability	Less portable (tied to OS)	Highly portable
	Cost	Higher	Lower
	Management	Complex (manage OS per VM)	Simpler (manage images/containers)
	Isolation	Stronger (full OS isolation)	Process isolation (kernel shared)

BENEFITS OF CONTAINERS

Containers provide a modern, efficient, and reliable way to build, ship, and run applications across any environment.



1. Consistency

Packages the app and dependencies together, running the same in dev, test, staging, and prod.



2. Faster Delivery

Lightweight images build quickly and start in seconds, speeding up CI/CD.



3. Resource Efficiency

Shares the host OS kernel, using fewer resources than virtual machines.



4. Scalability & Flexibility

Scale up or down by running more (or fewer) container instances dynamically.



5. Portability

Run containers on any system that supports Docker -- laptops, data centers, or clouds.



6. Isolation & Security

Each container is isolated from others and from the host, improving security posture.



7. Developer Productivity

Simplifies local development with consistent environments and easy dependency management.



8. High Density

Run more containers per server than VMs, enabling higher density and better ROI.

COMMON USE CASES

Microservices Architecture

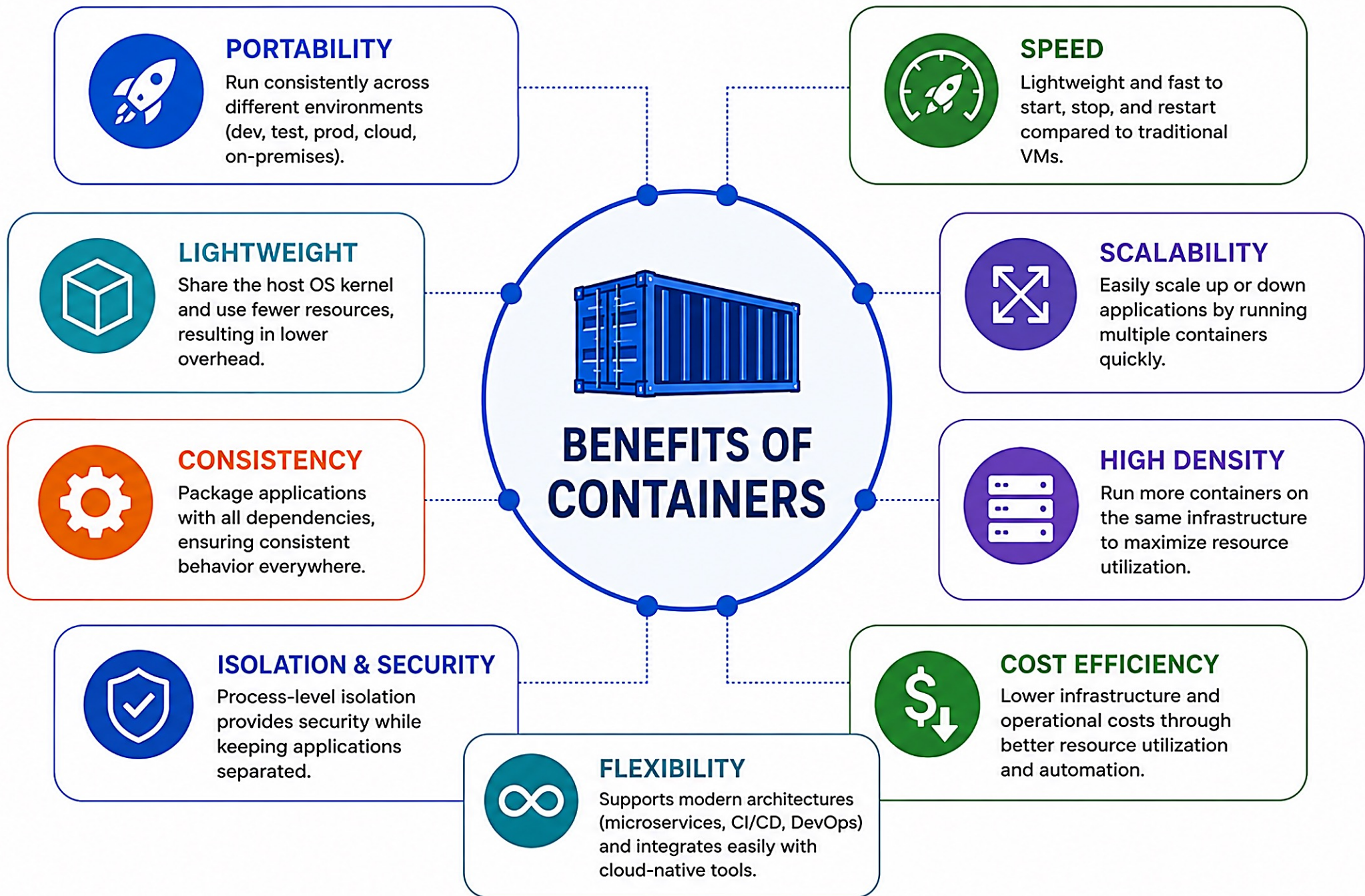
CI/CD Pipelines & Automation

Cloud-Native Applications

Legacy App Modernization

Dev/Test Environments

KEY TAKEAWAY Containers deliver consistency, speed, efficiency, and scalability -- enabling teams to build better software and operate with confidence.



CONTAINER ARCHITECTURE

Containers share the host OS kernel while providing isolated user spaces for applications -- lightweight, portable, efficient deployment.

IMAGE LAYERS (READ-ONLY)

- **Layer 1 -- Base Image:** the OS layer (e.g., Alpine, Ubuntu).
- **Layer 2 -- Libraries:** shared system libraries.
- **Layer 3 -- Dependencies:** application dependencies (e.g., JRE).
- **Layer N -- Application Code:** your built application (the JAR).

ISOLATION MECHANISM

- **Namespaces** — isolated view of PID, Network, IPC, Mount, UTS, and User resources per container.
- **Control Groups (cgroups)** — limit and allocate CPU, memory, I/O, and network resources to containers.

HOW IT WORKS

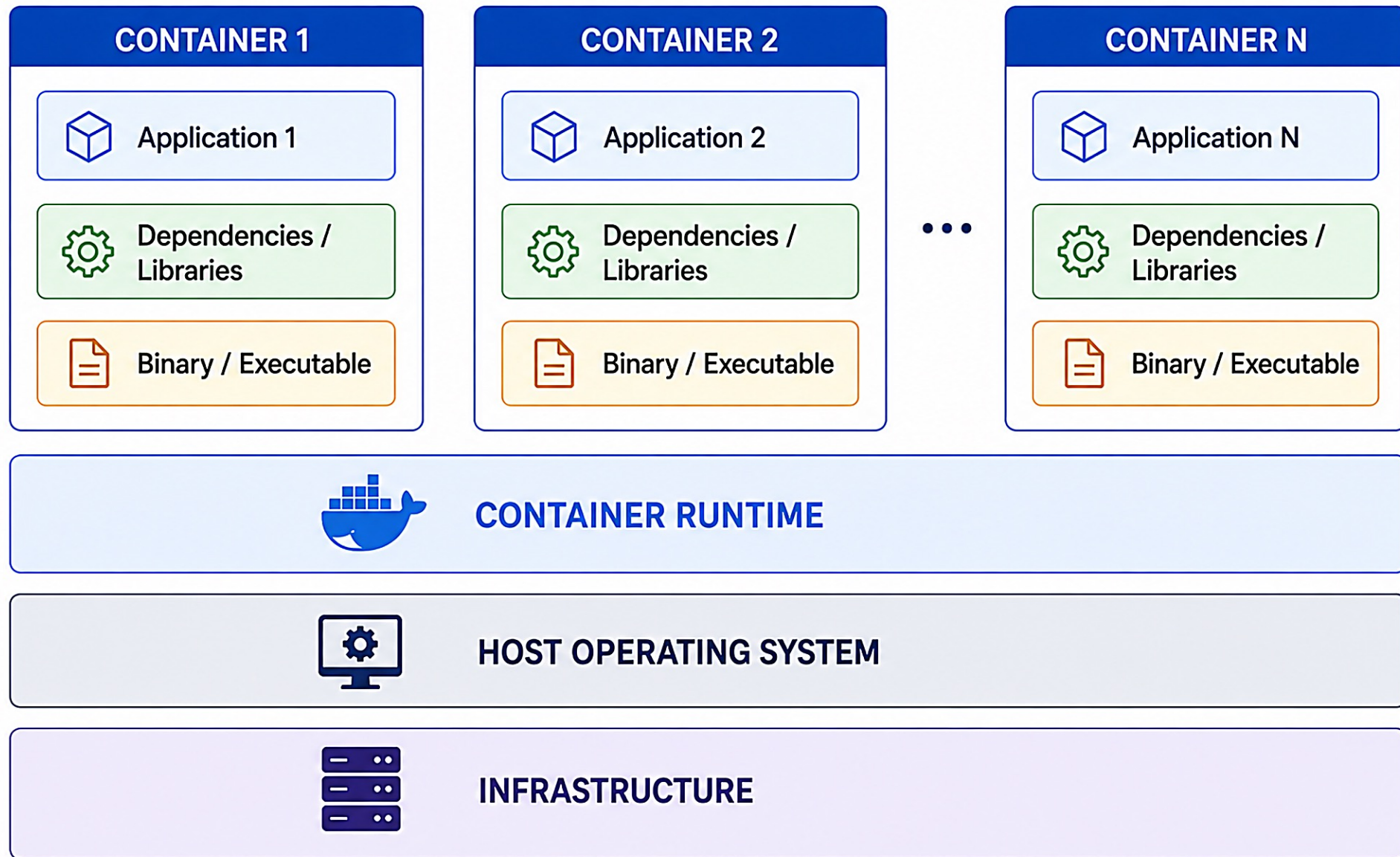


SHARED VS. ISOLATED

- **Shared** — Host OS kernel and physical hardware.
- **Isolated** — File system, process IDs, network, users, IPC, and mount points.

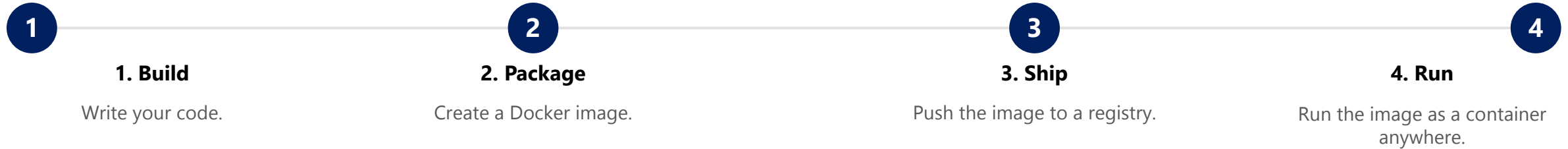
KEY TAKEAWAY Containers package your application and its dependencies, isolate it at the OS level, and share the host kernel -- delivering consistency, efficiency, and scalability.

Container Architecture



WHAT IS DOCKER?

Docker is an open-source platform that enables developers to build, package, ship, and run applications in containers.



KEY BENEFITS

- **Lightweight** — containers share the host OS kernel and start in seconds.
- **Portable** — build once, run anywhere -- laptop, data center, or cloud.
- **Fast** — speeds up development, testing, and CI/CD deployment.
- **Efficient** — uses fewer resources compared to virtual machines.
- **Isolated** — each container is isolated for better security and stability.

DOCKER USES

- **Microservices** — run each service in its own container.
- **CI/CD Pipelines** — consistent builds and reliable deployments.
- **Cloud Applications** — seamless portability to any cloud.
- **Development Environments** — reproducible, isolated dev setups.
- **Legacy App Modernization** — containerize and run legacy apps easily.

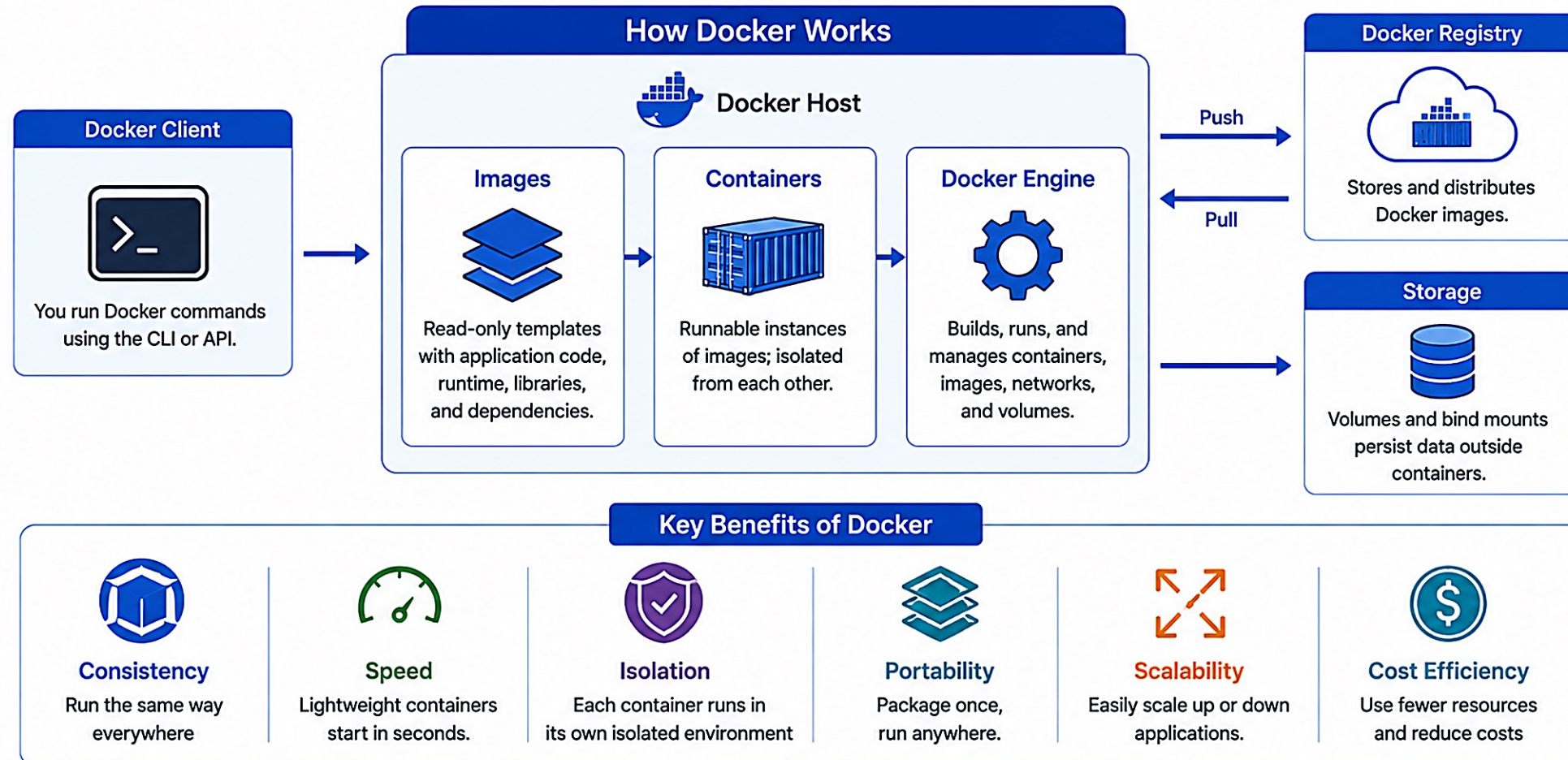
KEY TAKEAWAY Docker simplifies how we build, ship, and run applications by packaging everything into containers -- ensuring consistency, portability, and speed across the entire software lifecycle.

What is Docker?



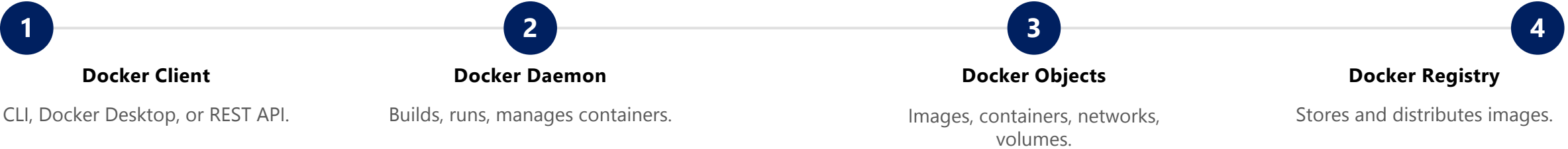
Docker is an open platform for developing, shipping, and running applications using containers.

It packages an application and all its dependencies into a standardized unit for software development.



DOCKER ARCHITECTURE OVERVIEW

Docker uses a client-server architecture. The Docker Client talks to the Docker Daemon (Engine), which builds, runs, and manages everything.



KEY HIGHLIGHTS

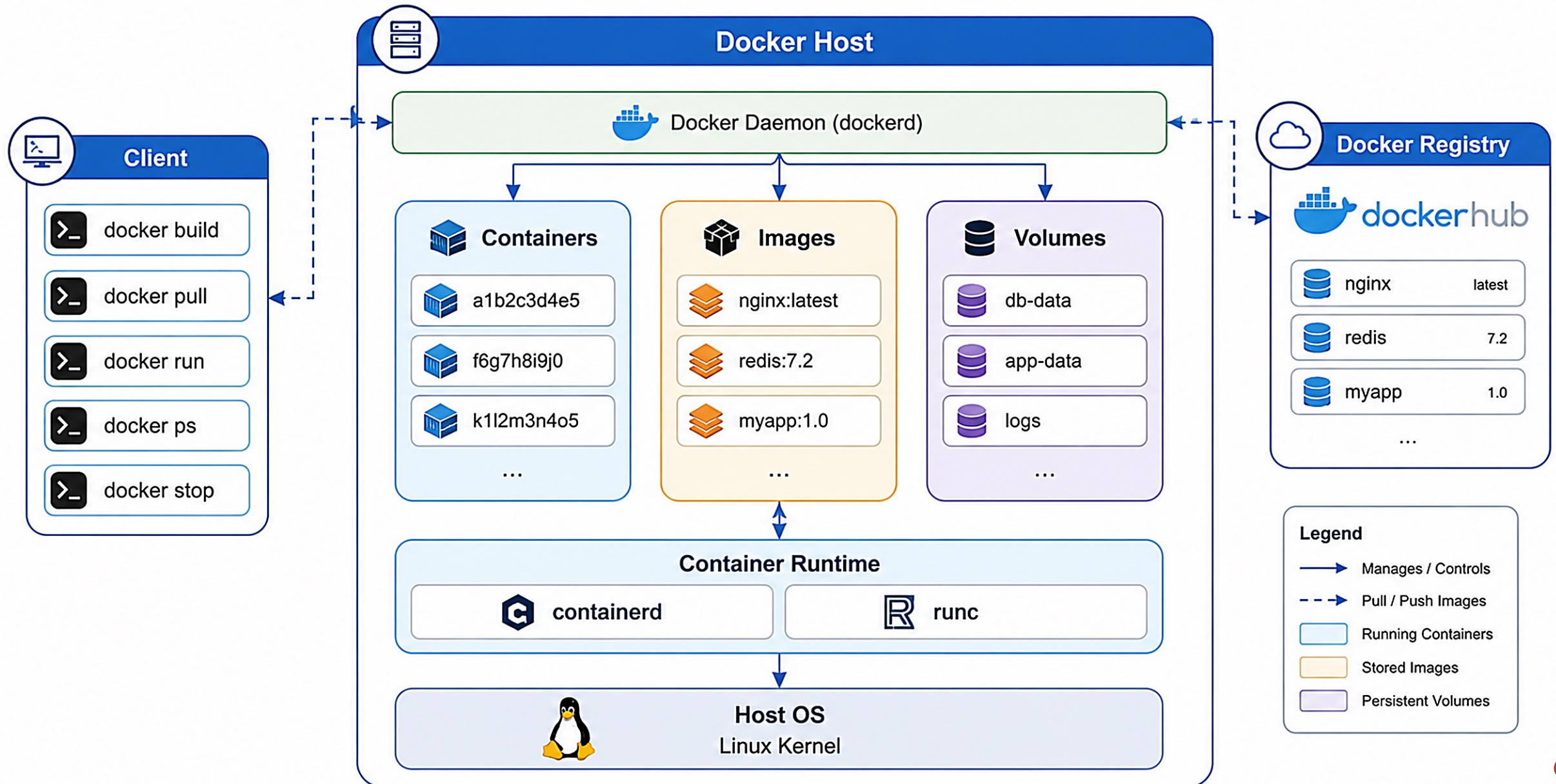
- The Client sends requests to the Docker Daemon (build, run, pull, push).
- The Daemon manages Docker objects: images, containers, networks, and volumes.
- The Registry stores and distributes Docker images (push to upload, pull to download).

OBJECTS MANAGED BY THE DAEMON

Object	Role
Images	Read-only templates used to create containers.
Containers	Running instances of images.
Networks	Enable communication between containers.
Volumes	Persistent data storage for containers.

KEY TAKEAWAY Docker's client-server architecture enables powerful, secure, and efficient management of containers. The Docker Daemon does the heavy lifting; the client makes it easy to use.

Docker Architecture Overview



DOCKER COMPONENTS

Docker is built from several key components that work together to build, ship, and run containerized applications.

Component	Responsibility
Docker Client	The user interface for Docker -- CLI, Docker Desktop, or REST API.
Docker Daemon (Engine)	The core engine: builds images, runs containers, manages networks and volumes.
Docker Image	A read-only template with instructions to create a Docker container.
Docker Container	A running instance of a Docker image -- isolated, portable, ephemeral.
Docker Registry	Stores and distributes Docker images (Docker Hub, private registry, AWS ECR).
Docker Volume	Persistent data storage for containers -- data survives container deletion.
Docker Network	Enables communication between containers and with the outside world.
Dockerfile	A text file with instructions to build a Docker image.
Docker Compose	Defines and runs multi-container applications using a YAML file.

ECOSYSTEM TOOLS THAT EXTEND DOCKER

Docker Desktop

Docker Hub

Docker Compose

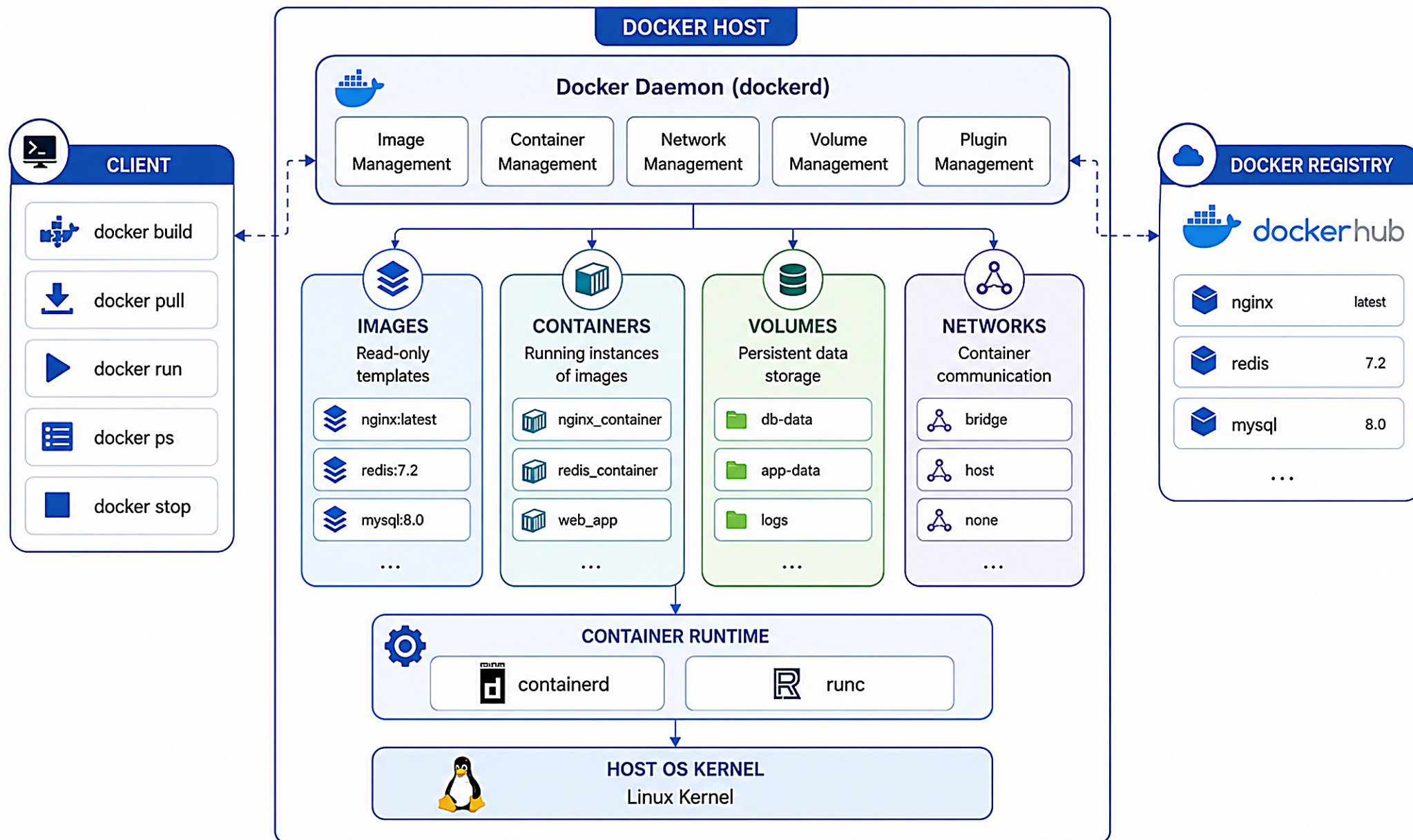
Docker Scan / Trivy

Docker Buildx

Docker Swarm

KEY TAKEAWAY Docker components work together to deliver containerized applications that are portable, consistent, and efficient across any environment.

Docker Components



DOCKER ENGINE

Docker Engine is the core of Docker -- a lightweight, high-performance container runtime and management service running as a background daemon.

KEY RESPONSIBILITIES

- Build and manage Docker images.
- Create, start, stop, and manage containers.
- Manage container networks.
- Manage persistent storage (volumes).
- Provide isolation via namespaces, cgroups, and other kernel features.
- Monitor container health and resource usage.

KEY POINTS

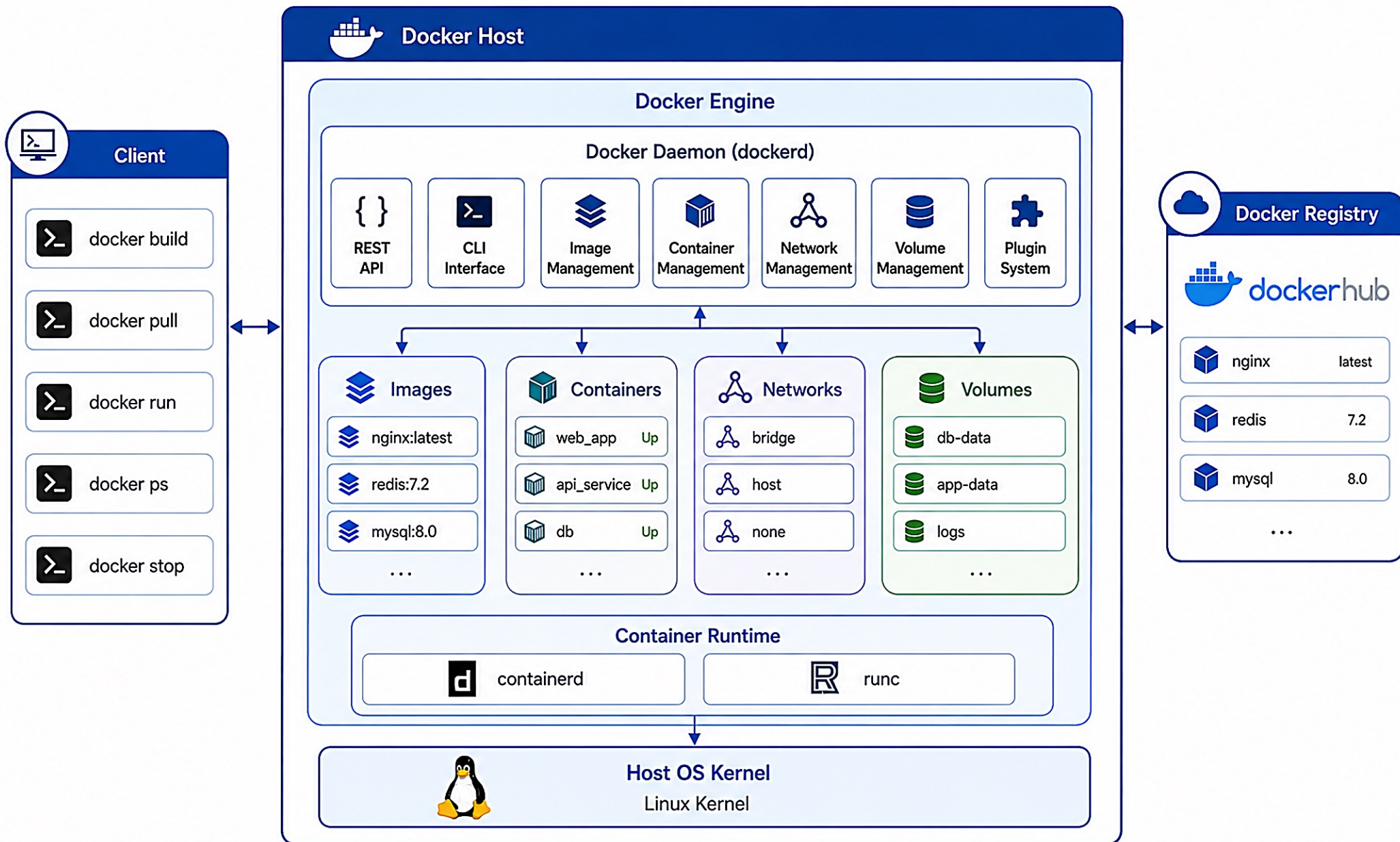
- Runs as a background service (daemon) and exposes a REST API used by Docker clients.
- Works on Linux, Windows, and macOS.

HOW DOCKER ENGINE WORKS (FLOW)



KEY TAKEAWAY Docker Engine is the heart of Docker. It exposes a powerful API and manages all aspects of containers, images, networks, and volumes to deliver applications consistently and efficiently.

Docker Engine



UNDERSTANDING DOCKER IMAGES

A Docker Image is a lightweight, standalone, read-only template containing everything needed to run an application: code, runtime, libraries, and config.

IMAGE LAYERS (BOTTOM UP)

- Base OS (kernel).
- Base Image Layer (e.g., Alpine, Temurin JRE).
- Runtime Layer (e.g., Java, JRE).
- Dependencies (libraries, packages).
- Application Layer (your code -- the JAR).

KEY POINTS

- Images are immutable and read-only.
- Built in layers for efficiency -- unchanged layers are reused across builds.
- Versioned and shareable via registries.

IMAGE VS. CONTAINER

Image	Container
Blueprint	Running instance
Read-only	Read-write layer
Built once	Created from image
Stored in registry	Ephemeral

Example: Simple Spring Boot Image

```
FROM eclipse-temurin:17-jre-alpine
# Base image

WORKDIR /app
# Set working directory

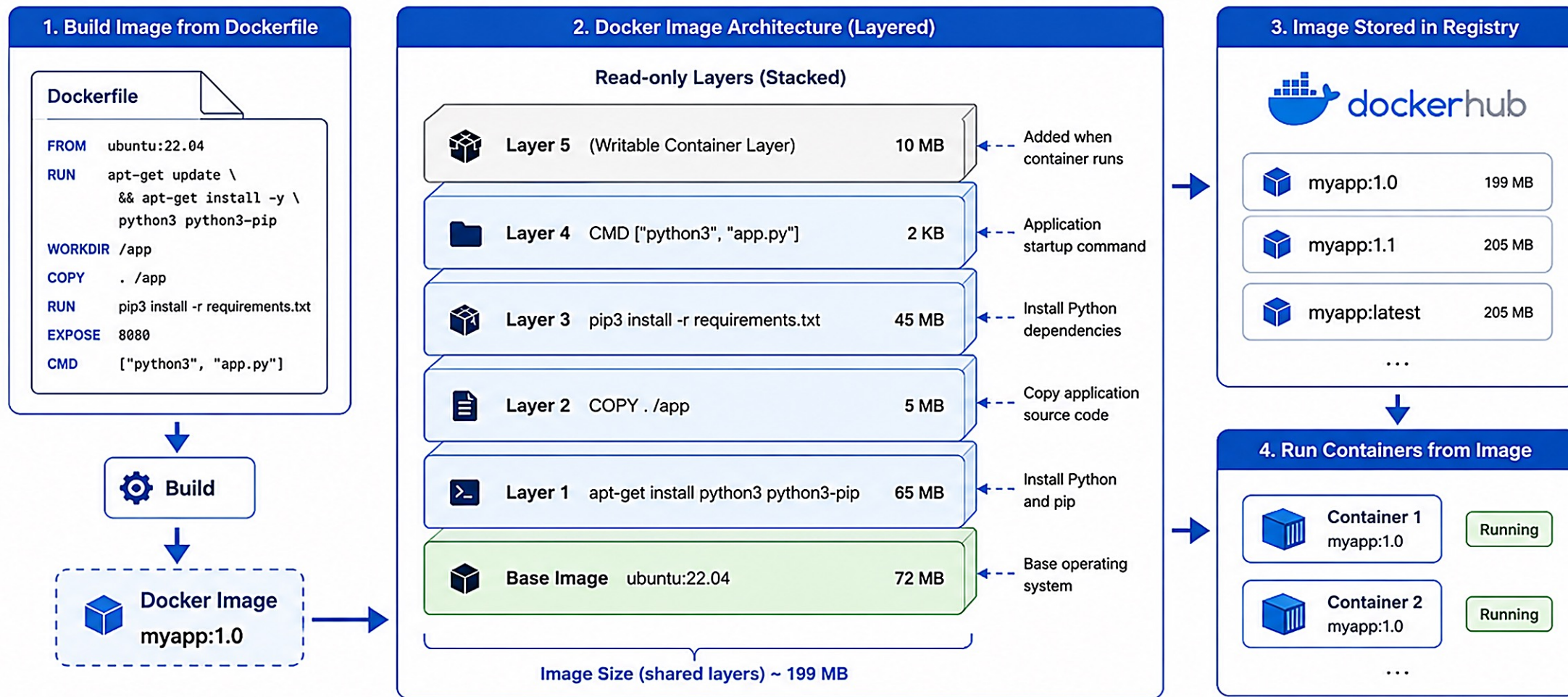
COPY target/customer-service.jar app.jar
# Copy application

EXPOSE 8080
# Expose port

ENTRYPOINT ["java", "-jar", "app.jar"]
# Run application
```

KEY TAKEAWAY Docker images are the foundation of containerization. They package your application and dependencies into immutable, layered templates that can be shared and run consistently anywhere.

Understanding Docker Images



Key Characteristics of Docker Images



Layered

Images are built in layers. Unchanged layers are shared and reused.



Immutable

Once built, images cannot be changed. Create a new image for updates.



Shareable

Images can be easily shared via registries like Docker Hub.



Versioned

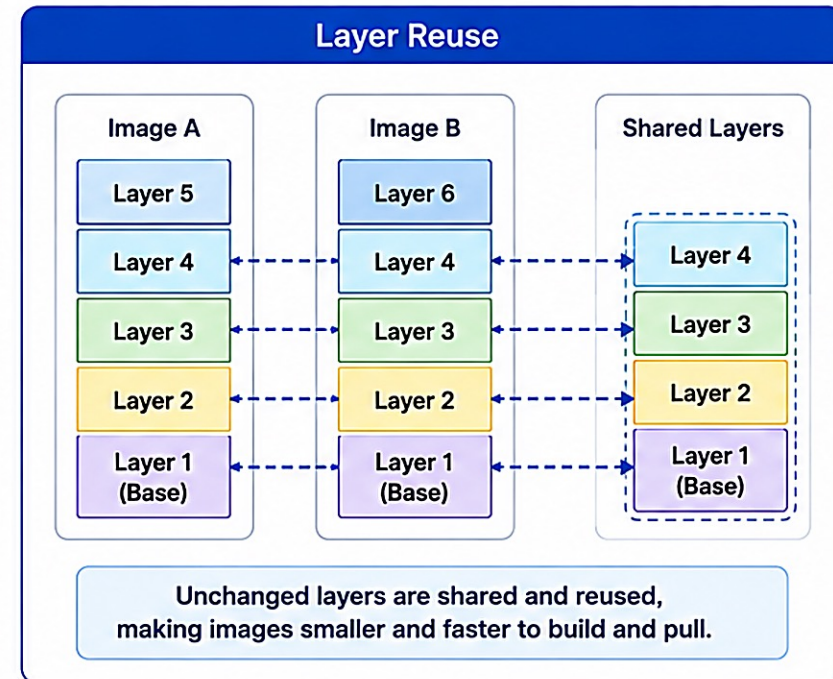
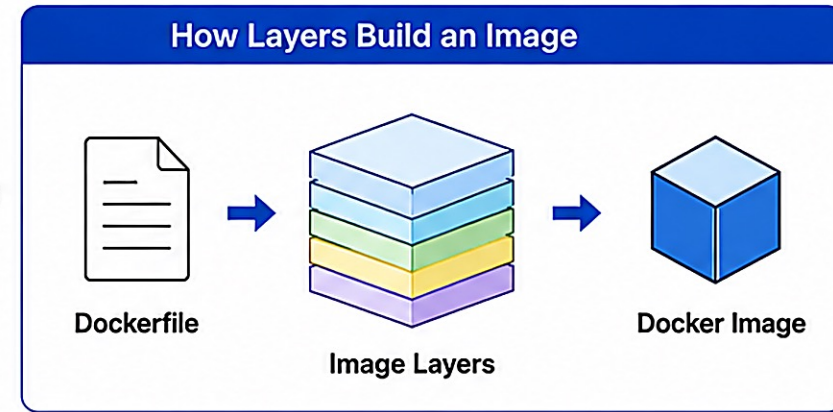
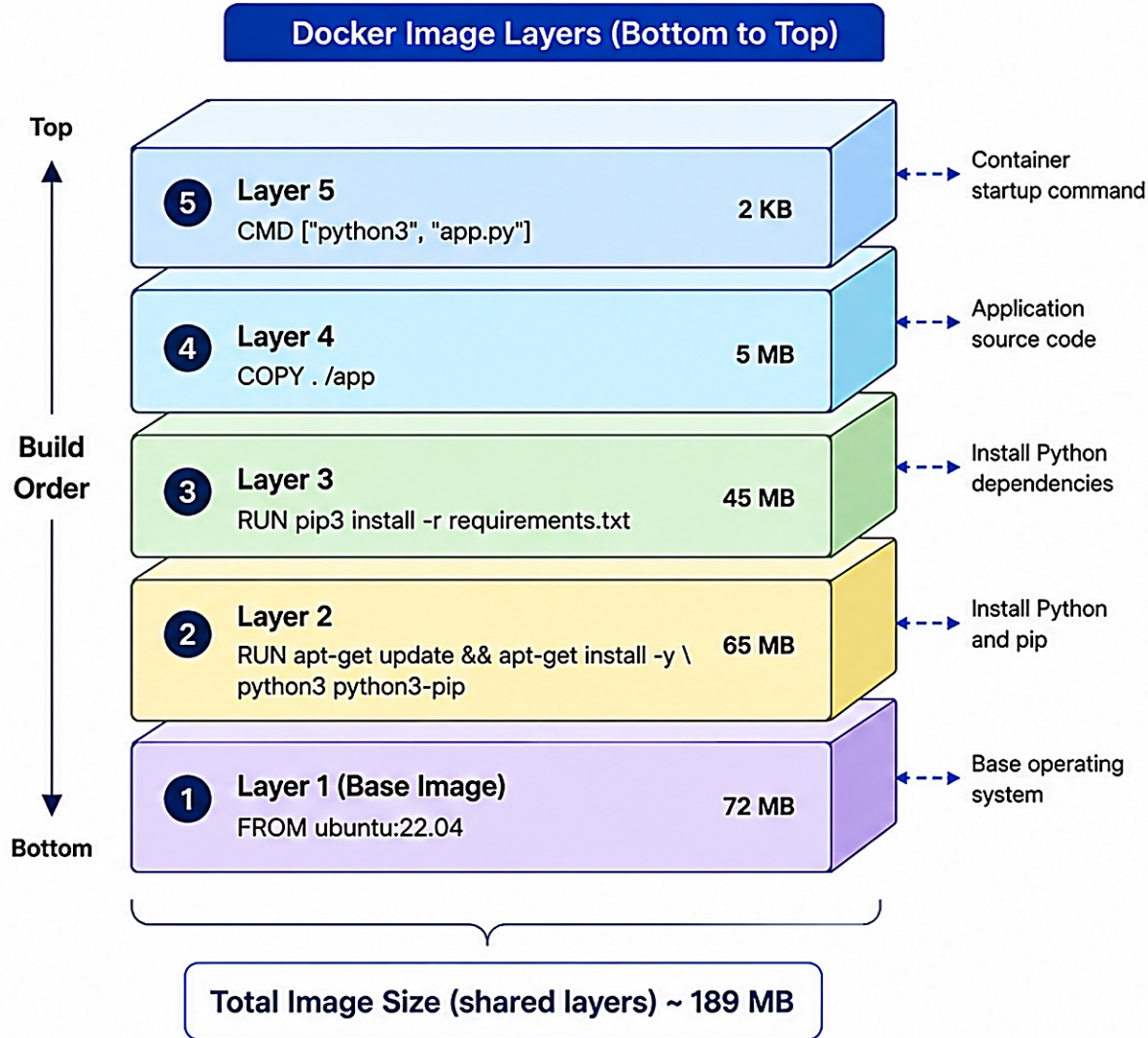
Use tags (e.g., 1.0, 1.1, latest) to manage image versions.



Efficient

Layer sharing reduces size and speeds up builds and pulls.

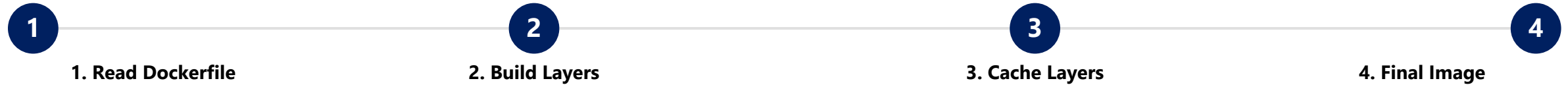
Image Layers



BUILDING DOCKER IMAGES

Building a Docker image means creating a read-only template that contains everything needed to run your application in a container.

HOW IMAGE BUILDING WORKS



BUILD CONTEXT

- **Included:** source code, JAR/WAR files, configuration files, static resources.
- **Excluded via .dockerignore:** target/, .git/, *.log, node_modules/, secrets, IDE files.

Build commands

```
docker build -t customer-service:1.0 .
docker images
docker rmi customer-service:1.0
```

COMMON DOCKERFILE INSTRUCTIONS

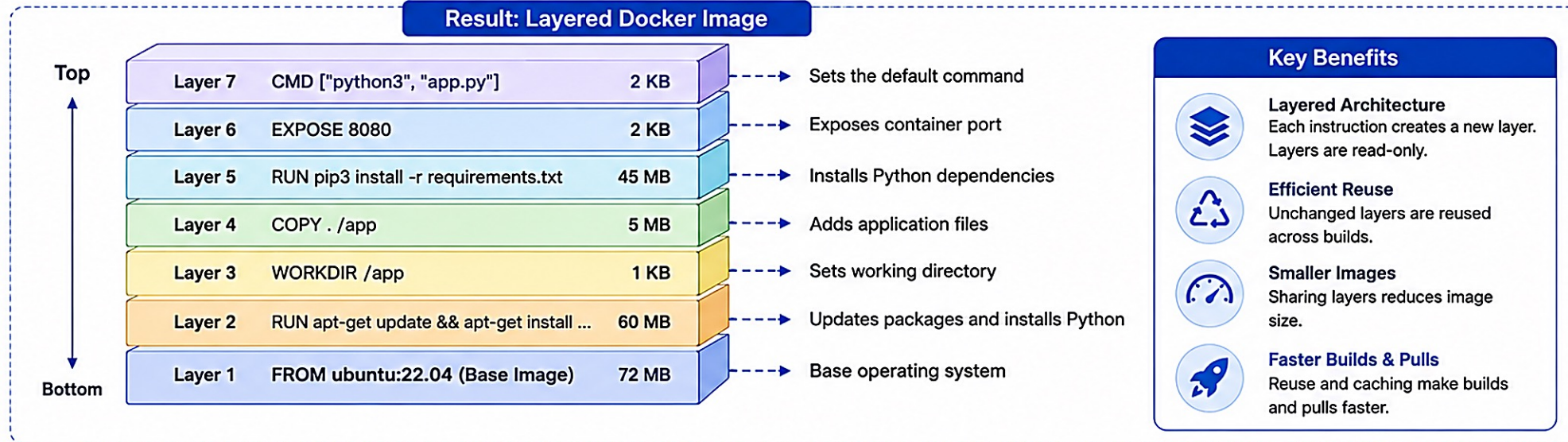
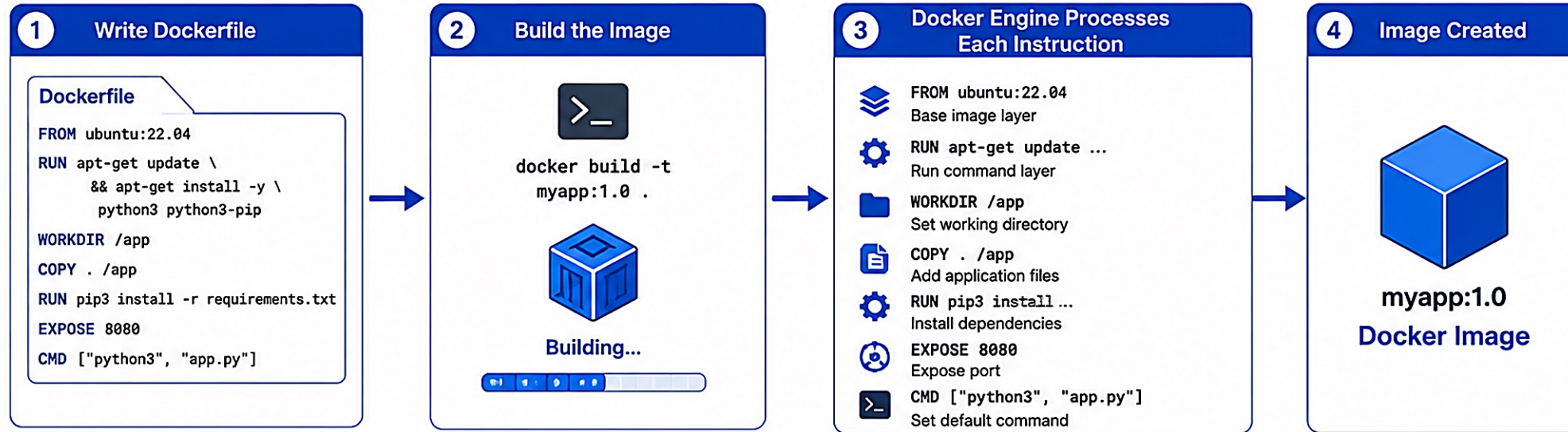
Instruction	Purpose
FROM	Base image.
WORKDIR	Set working directory.
COPY	Copy files from host.
RUN	Execute build commands.
EXPOSE	Document the listening port.
ENTRYPOINT / CMD	Configure container startup.

BEST PRACTICES

- Start with a minimal base image.
- Leverage layer caching (order instructions carefully).
- Use .dockerignore to exclude unnecessary files.
- Run containers as non-root users.

KEY TAKEAWAY A well-written Dockerfile and efficient build process produce small, secure, and portable images that can run anywhere consistently.

Building Docker Images



Build Command: `docker build -t myapp:1.0 .`



Image Created: `myapp:1.0`

IMAGE OPTIMIZATION TECHNIQUES (1 OF 2)

Optimizing Docker images reduces size, improves security, speeds up builds, and lowers storage and network costs.

1

Minimal Base Image

Use official, minimal base images (Alpine, distroless, or slim variants) instead of full JDK images.

2

Multi-Stage Builds

Build in one stage; copy only what's needed (the JAR) into a clean, lightweight runtime image.

3

Minimize Layers

Combine related RUN commands and clean up in the same layer to keep images small.

4

Leverage .dockerignore

Exclude files and folders not needed in the build, reducing context size and build time.

5

Order Instructions Smartly

Place least-frequently-changed instructions first (deps before source) to maximize cache hits.

IMAGE SIZE IMPACT EXAMPLE

Base Image Approach	Approximate Size
Default JDK (openjdk:17)	~470 MB
Slim JRE (temurin:17-jre-slim)	~180 MB
Alpine JRE (temurin:17-jre-alpine)	~80 MB
Distroless	~60 MB

KEY TAKEAWAY A minimal base image plus multi-stage builds are the two highest-impact optimizations -- together they can cut image size by 80-90%.

Image Optimization Techniques

1 Use Minimal Base Images

Avoid large base images



Impact: Smaller base image reduces overall image size

2 Combine RUN Commands

Reduce layers by combining commands

Bad (Multiple Layers)

```
RUN apt-get update
RUN apt-get install -y python3
RUN apt-get install -y curl
```



Good (Single Layer)

```
RUN apt-get update && \
  apt-get install -y python3 curl && \
  rm -rf /var/lib/apt/lists/*
```



Impact: Fewer layers, smaller image, faster build

3 Clean Up in Same Layer

Remove unnecessary files in the same RUN layer

Bad

```
RUN apt-get install -y build-essential
RUN rm -rf /var/lib/apt/lists/*
```



Good

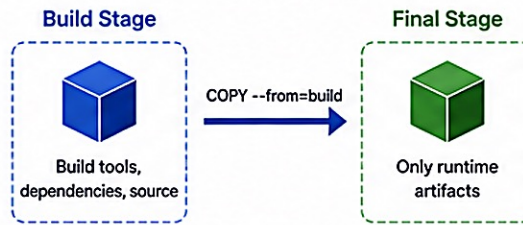
```
RUN apt-get install -y build-essential && \
  rm -rf /var/lib/apt/lists/*
```



Impact: Avoids keeping unnecessary files in intermediate layers

4 Use Multi-Stage Builds

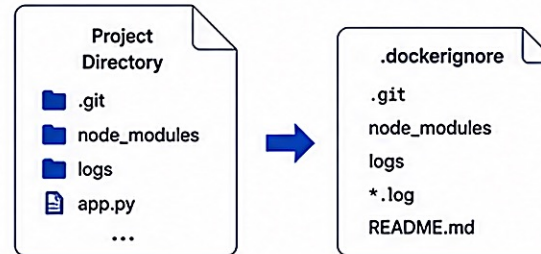
Copy only what you need from build stage



Impact: Excludes build tools and caches from final image

5 Leverage .dockerignore

Exclude files and folders from build context



Impact: Smaller build context, faster builds, smaller image

6 Remove Unnecessary Packages

Install only what's necessary

Bad

```
RUN apt-get install -y \
  vim curl wget git build-essential \
  telnet net-tools
```



Good

```
RUN apt-get install -y \
  ca-certificates curl
```



Impact: Fewer packages = smaller and more secure image

Best Practices Checklist



Start with minimal base images



Combine commands to reduce layers



Clean up in the same layer



Use multi-stage builds



Use .dockerignore effectively



Install only what's necessary



Continuously analyze and optimize

IMAGE OPTIMIZATION TECHNIQUES (2 OF 2)

Five more techniques for smaller, faster, more secure images -- plus a best-practice checklist to apply them consistently.

6

Remove Unnecessary Packages

Install only what you need -- avoid apt-get install of large package bundles.

7

Use Specific Tags, Not :latest

Pin a specific version tag (e.g., 21-jre-alpine) for reproducible, stable builds.

8

Use Distroless (When Possible)

Distroless images contain only the app and runtime -- no package manager, no shell.

9

Compress & Clean Temp Files

Clean package caches, temp files, and build artifacts in the same layer they were created.

10

10. Scan Images Regularly

Scan images for vulnerabilities before pushing to a registry (Trivy, Docker Scout, Gype, Snyk).

BEST PRACTICE CHECKLIST

- Use minimal base images and multi-stage builds.
- Minimize layers; order instructions for caching; use .dockerignore.
- Remove unnecessary packages; use specific tags, not :latest.
- Prefer distroless where possible; scan images for vulnerabilities regularly.

KEY TAKEAWAY A well-optimized Docker image is smaller, faster, more secure, and more efficient. Follow these techniques to build production-ready container images.

DOCKERFILE OVERVIEW

A Dockerfile is a text file containing instructions used to build a Docker image. Each instruction creates a layer in the image.

Example Dockerfile (Spring Boot application)

```
# Use a lightweight base image with JRE
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

COPY target/customer-service.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

WHAT THIS DOCKERFILE DOES

- Uses a minimal base image with a Java 17 JRE.
- Sets the working directory inside the container.
- Copies the built JAR file into the image.
- Exposes port 8080 (documentation only -- does not publish it).
- Runs the Spring Boot application on container start.

DOCKERFILE BUILD CONTEXT

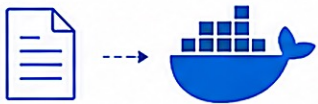
- **Sent to Docker:** Dockerfile, source code, pom.xml, resources, static files.
- **Excluded (.dockerignore):** .git/, node_modules/, target/, *.log, secrets.

KEY TAKEAWAY A well-written Dockerfile is the foundation of a great container image. It ensures repeatability, efficiency, and portability.

Dockerfile Overview

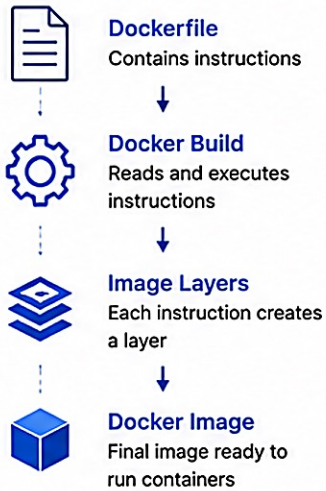
A Dockerfile is a text file that contains instructions to build a Docker image.

What is a Dockerfile?



A Dockerfile is a blueprint for building Docker images automatically.

How it Works



Dockerfile Example

```
1  # Use an official Python runtime as a parent image
2  FROM python:3.11-slim
3
4  # Set the working directory in the container
5  WORKDIR /app
6
7  # Copy requirements file and install dependencies
8  COPY requirements.txt .
9  RUN pip install --no-cache-dir -r requirements.txt
10
11 # Copy application source code
12 COPY . .
13
14 # Expose application port
15 EXPOSE 8080
16
17 # Set default command
18 CMD ["python", "app.py"]
```

Annotations for the example Dockerfile:

- Line 2: FROM python:3.11-slim → Use base image
- Line 5: WORKDIR /app → Set working directory
- Line 8: COPY requirements.txt . → Copy dependency file
- Line 9: RUN pip install --no-cache-dir -r requirements.txt → Install dependencies
- Line 12: COPY . . → Copy application code
- Line 15: EXPOSE 8080 → Expose port
- Line 18: CMD ["python", "app.py"] → Default command to run

Common Instructions

	FROM	Set base image
	WORKDIR	Set working directory
	COPY	Copy files or directories
	ADD	Copy with extra features (e.g., URL, extract)
	RUN	Run command in image build
	EXPOSE	Expose port
	ENV	Set environment variable
	CMD	Default command
	ENTRYPOINT	Configure container entry point
	USER	Set user
	VOLUME	Create mount point
	ARG	Build-time variable
	LABEL	Add metadata

Build and Run



Best Practices



Use minimal base images
Smaller images, fewer vulnerabilities.



Order matters
Put stable layers (deps) earlier for better caching.



Combine RUN commands
Reduce layers by combining commands.



Clean up
Remove unnecessary files and cache.



Use .dockerignore
Exclude unnecessary files from build context.



Keep it simple
Clear, readable, and maintainable Dockerfiles.

COMMON DOCKERFILE INSTRUCTIONS

Dockerfile instructions are the building blocks used to create images -- executed in order, each performing one specific task.

Instruction	Purpose
FROM	Sets the base image for the build.
WORKDIR	Sets the working directory for later instructions.
COPY	Copies files or directories from host to image.
ADD	Like COPY, but can extract archives and fetch URLs.
RUN	Executes a command in a new build-time layer.
EXPOSE	Documents the port the container listens on.

Instruction	Purpose
ENV	Sets environment variables inside the image.
ENTRYPOINT	Configures the container's main executable.
CMD	Provides default arguments for ENTRYPOINT.
VOLUME	Creates a mount point for persistent data.
USER	Sets the (ideally non-root) user to run the container as.
HEALTHCHECK	Defines a command Docker uses to check container health.

INSTRUCTION CATEGORIES



KEY TAKEAWAY Mastering these Dockerfile instructions helps you create smaller, secure, and efficient Docker images -- and every one of them appears in Lab 41's own Dockerfile.

Common Dockerfile Instructions

Instruction	Purpose	Example	Example Usage	Notes
 FROM	Sets the base image for subsequent instructions.	<code>FROM <image>:<tag></code>	<pre># Use official Python image FROM python:3.11-slim</pre>	 Must be the first instruction.
 WORKDIR	Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions.	<code>WORKDIR <path></code>	<pre># Set working directory WORKDIR /app</pre>	 Creates the directory if it doesn't exist.
 COPY	Copies files or directories from the build context to the image.	<code>COPY <src> <dest></code>	<pre># Copy file to image COPY app.py /app/</pre>	 Cannot download files from URLs.
 ADD	Copies files or directories and can also download/extract remote resources.	<code>ADD <src> <dest></code>	<pre># Download and extract a tar file ADD https://example.com/file.tar.gz /app/</pre>	 Supports URLs and auto-extraction of archives.
 RUN	Executes commands in a new layer during image build.	<code>RUN <command></code>	<pre># Install packages RUN apt-get update && \ apt-get install -y python3</pre>	 Each RUN creates a new image layer.
 EXPOSE	Informs Docker that the container listens on the specified network ports at runtime.	<code>EXPOSE <port> [<port>/protocol]</code>	<pre># Expose port 8080 EXPOSE 8080</pre>	 Does not publish the port. Use -p with docker run.
 ENV	Sets environment variables.	<code>ENV <key>=<value></code>	<pre># Set environment variable ENV APP_ENV=production</pre>	 Available in subsequent instructions and runtime.
 CMD	Sets the default command to run when a container starts.	<code>CMD ["executable","param"]</code> or <code>CMD command param</code>	<pre># Default command CMD ["python", "app.py"]</pre>	 Can be overridden by docker run command.
 ENTRYPOINT	Configures a container that will run as an executable.	<code>ENTRYPOINT ["executable","param"]</code>	<pre># Set entrypoint ENTRYPOINT ["python", "app.py"]</pre>	 Not easily overridden. CMD provides default arguments.
 USER	Sets the user (and optional group) to run commands as.	<code>USER <user>[:<group>]</code>	<pre># Run as non-root user USER appuser</pre>	 Improves security by avoiding root.
 VOLUME	Creates a mount point with the specified name and marks it as holding externally mounted volumes.	<code>VOLUME ["/data"]</code>	<pre># Create volume VOLUME ["/data"]</pre>	 Data persists even if container is removed.
 ARG	Defines build-time variables.	<code>ARG <name>[=<default>]</code>	<pre># Build argument ARG VERSION=1.0.0</pre>	 Not available at runtime.
 LABEL	Adds metadata to an image.	<code>LABEL <key>=<value></code>	<pre># Add metadata LABEL maintainer="devops@example.com"</pre>	 Useful for image information.

MULTI-STAGE BUILDS

Multi-stage builds use multiple FROM statements in one Dockerfile -- copy only the artifacts you need into a smaller, more secure final image.

A full build stage (Maven + JDK) compiles the JAR; only the JAR is copied into a lean runtime stage (JRE-only, non-root). This is the exact pattern Lab 41's `crm-api` Dockerfile uses.

REAL DOCKERFILE -- LAB 41 `crm-api` (multi-stage, non-root, health-checked)

```
# syntax=docker/dockerfile:1
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml .
COPY .mvn .mvn
COPY mvnw mvnw
RUN chmod +x mvnw && ./mvnw -B -q -DskipTests dependency:go-offline
COPY src ./src
RUN ./mvnw -B clean verify

FROM eclipse-temurin:21-jre
RUN useradd --system --uid 10001 --create-home spring
WORKDIR /app
COPY --from=build --chown=spring:spring /workspace/target/*-SNAPSHOT.jar app.jar
USER 10001
EXPOSE 8080
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75"
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
  CMD wget -qO- http://127.0.0.1:8080/actuator/health/readiness || exit 1
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

KEY TAKEAWAY Multi-stage builds cut image size dramatically (source example: ~780 MB single-stage to ~115 MB multi-stage, ~85% smaller) while keeping build tools out of the runtime image.

Multi-Stage Builds

Use multiple stages in a single Dockerfile to build smaller, faster, and more secure images.

What & Why

Multi-stage builds use multiple FROM statements in a Dockerfile. Each stage has its own filesystem and only artifacts you choose are copied to the final image.



Why Use Multi-Stage Builds?



Smaller Images

Only contain the runtime dependencies you need.



Improved Security

Exclude build tools and sensitive files.



Faster Builds & Transfers

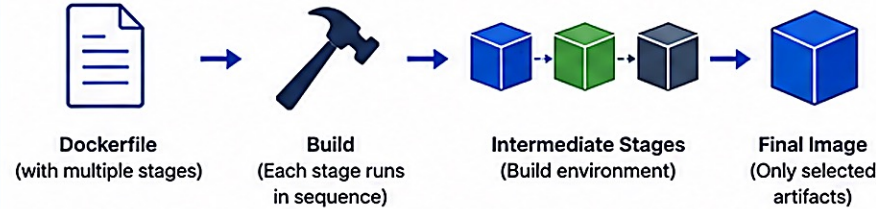
Less data to build, push and pull.



Better Separation

Keep build environment separate from runtime.

How Multi-Stage Builds Work



Key Concept

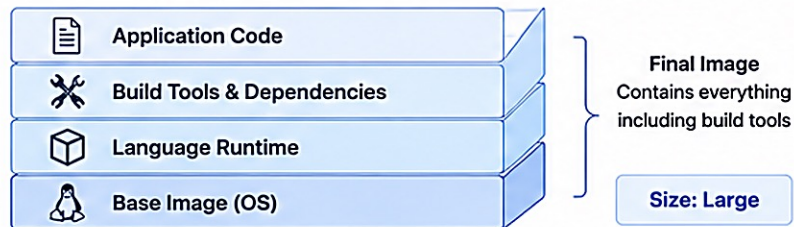
Artifacts (files, binaries, etc.) are copied from one stage to another using the `COPY --from=<stage>` instruction.

Dockerfile Example

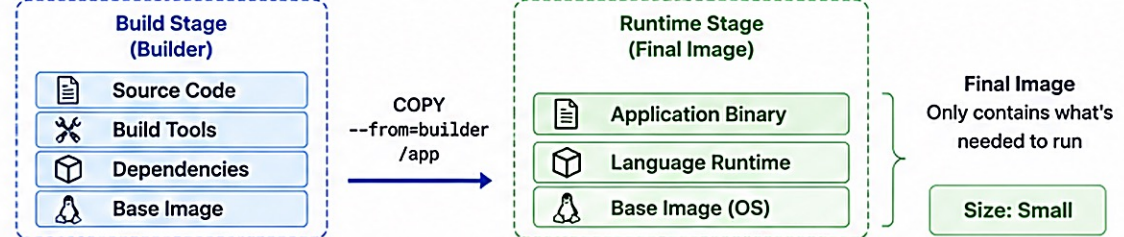
```
1 # ----- Build Stage -----
2 FROM golang:1.22 AS builder
3 WORKDIR /src
4 COPY go.mod go.sum ./
5 RUN go mod download
6 COPY . .
7 RUN CGO_ENABLED=0 go build -o app .
8
9 # ----- Runtime Stage -----
10 FROM alpine:3.19
11 WORKDIR /app
12
13 # Copy only the binary from builder stage
14 COPY --from=builder /src/app ./app
15
16 EXPOSE 8080
17
18 # Run the application
19 CMD ["/app"]
20
```

Single-Stage Build vs Multi-Stage Build

Single-Stage Build (Larger Image)



Multi-Stage Build (Smaller Image)



Key Benefits of Multi-Stage Builds



Smaller Images

Reduced size, lower storage and bandwidth costs.



Enhanced Security

No build tools or secrets in final image.



Better Performance

Faster build, push and pull times.



Cleaner Dockerfiles

Well-structured and easier to maintain.



Production Ready

Lean, efficient and optimized for production.

TRY IT YOURSELF — EXERCISE 1: MULTI-STAGE SKETCH

Reinforces: separating the Maven build stage from the JRE runtime stage on paper, before writing a real Dockerfile.

GOAL: notes/lab41-multistage-sketch.md

Stage	Must Contain	Must NOT Contain
build	JDK 21, Maven, sources	Runtime secrets
runtime	JRE, app JAR, non-root user	Maven, .git, passwords

STEPS

- **1. Stages** — name build (Maven + JDK 21) and runtime (JRE 21); list what copies between them (the JAR only).
- **2. Check the reference** — runtime must not include Maven, source, or .git.
- **3. User** — plan a non-root UID (e.g. 10001) and note why root fails the lab.
- **4. CRM note** — fixtures CUS-1001 / CUS-1002 are app data at runtime -- not build args.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-multistage-sketch.md (workspace: examples/module-41-exercises).

DOCKERFILE BEST PRACTICES (1 OF 2)

Following best practices helps you build images that are small, secure, fast, and easy to maintain.

1 Minimal Base Images

Use official, lightweight base images (Alpine, distroless, slim).

2 Multi-Stage Builds

Build in one stage; copy only necessary artifacts into the final image.

3 Order Instructions Smartly

Put least-frequently-changing instructions first to leverage the cache.

4 Combine Commands

Combine related commands to reduce the number of layers.

5 Clean Up Unnecessary Files

Remove package caches and temp files to keep images small.

6 Use .dockerignore

Exclude files/folders not needed in the build context.

7 Pin Versions

Pin base image and dependency versions for reproducible builds.

8 Use Specific Tags, Not :latest

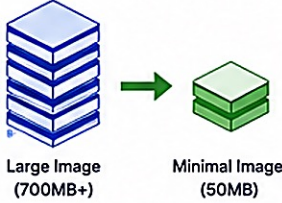
Avoid the :latest tag in production for predictable builds.

KEY TAKEAWAY Well-written Dockerfiles lead to smaller, secure, faster, and more maintainable container images.

Dockerfile Best Practices

1 Use Minimal Base Images

Start with the smallest base image you need.



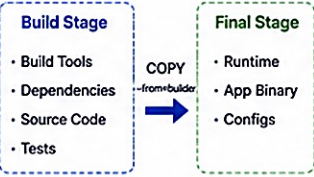
Large Image (700MB+)

Minimal Image (50MB)

Why?
Smaller images, fewer vulnerabilities, faster pulls.

2 Leverage Multi-Stage Builds

Use multi-stage builds to keep only what's needed.



Build Stage

- Build Tools
- Dependencies
- Source Code
- Tests

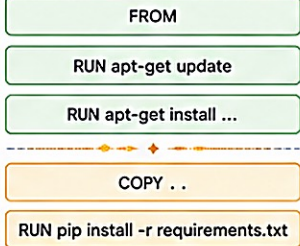
Final Stage

- Runtime
- App Binary
- Configs

Why?
Smaller final images and improved security.

3 Order Instructions from Stable to Volatile

Place less frequently changed instructions first.



FROM

RUN apt-get update

RUN apt-get install ...

COPY ..

RUN pip install -r requirements.txt

Why?
Better layer caching and faster rebuilds.

4 Combine RUN Commands

Combine commands to reduce the number of layers.

Instead of:

```
RUN apt-get update
RUN apt-get install -y curl \
git \
RUN rm -rf /var/lib/apt/lists/*
```

Use:

```
RUN apt-get update && \
apt-get install -y curl git && \
rm -rf /var/lib/apt/lists/*
```

Why?
Fewer layers, smaller image size, cleaner history.

5 Clean Up in Same Layer

Remove unnecessary files in the same layer.

Bad:

```
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
```

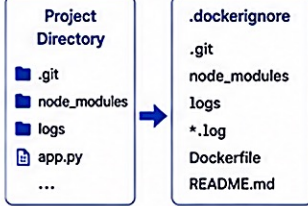
Good:

```
RUN apt-get install -y curl && \
rm -rf /var/lib/apt/lists/*
```

Why?
Avoid leaving unnecessary files in intermediate layers.

6 Use .dockerignore File

Exclude unnecessary files from the build context.



Project Directory

- .git
- node_modules
- logs
- app.py
- ...

.dockerignore

- .git
- node_modules
- logs
- *.log
- Dockerfile
- README.md

Why?
Smaller build context, faster builds, less data sent.

7 Pin Base Image Versions

Use specific tags, not latest.

Bad:

```
FROM ubuntu:latest
```

Good:

```
FROM ubuntu:22.04
```

Why?
Consistent, reproducible builds and avoids unexpected changes.

8 Use Non-Root User

Run containers as a non-root user whenever possible.

Bad:

```
USER root
```

Good:

```
USER appuser
```



Why?
Improved security and least privilege.

9 Set Environment Variables

Use ENV for configuration and defaults.

```
ENV APP_ENV=production
ENV LOG_LEVEL=info
ENV PORT=8080
```



Why?
Flexible and easier to configure at runtime.

10 Expose Ports

Document the port the application listens on.

```
EXPOSE 8080
```



Why?
Better documentation and self-explanatory images.

11 Use HEALTHCHECK

Add a health check to your image.

```
HEALTHCHECK --interval=30s \
--timeout=5s --retries=3 \
CMD curl -f http://localhost:8080/health \
|| exit 1
```



Why?
Helps orchestrators detect unhealthy containers.

12 Keep It Simple and Readable

Write clear, concise Dockerfiles with comments.

```
# Install dependencies
RUN apt-get update && \
apt-get install -y curl

# Copy application
COPY ..

# Run the app
CMD ["python", "app.py"]
```

Why?
Easier to maintain, review, and collaborate.



Minimize the number of layers



Keep images small and clean



Prioritize security



Optimize for build performance



Document intent clearly



Continuously review and improve

DOCKERFILE BEST PRACTICES (2 OF 2)

The remaining seven practices -- including the two most security-critical ones for Lab 41: non-root users and health checks.



Run as Non-Root User

Improve container security by running as a fixed non-root user (e.g., UID 10001).



10. Set a Healthcheck

Define a HEALTHCHECK so Docker can detect container issues early.



11. Use WORKDIR

Set a working directory to avoid relative-path guesswork.



12. Use COPY Over ADD

COPY is predictable and easier to understand than ADD (unless you truly need it).



13. Expose Only Necessary Ports

Only EXPOSE the ports your application actually uses.



14. Document Your Dockerfile

Add comments explaining important steps and decisions.



15. Keep It Small & Focused

One Dockerfile per application -- avoid unrelated concerns in one file.

KEY TAKEAWAY Build small images. Build secure images. Build fast, reliable images. Build with consistency -- this is exactly what Lab 41's Dockerfile is graded on.

TRY IT YOURSELF — EXERCISE 2: FILL DOCKERFILE TODOS

Reinforces: every instruction and best practice from this section, applied by filling in a real Dockerfile skeleton.

GOAL: notes/lab41-dockerfile-todos.md

Dockerfile skeleton (fill every blank)

```
FROM _____ AS build
WORKDIR /workspace
COPY _____ .
RUN ./mvnw -B -DskipTests package
FROM _____ AS runtime
USER _____
COPY --from=build _____ /app/app.jar
HEALTHCHECK CMD _____
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

STEPS

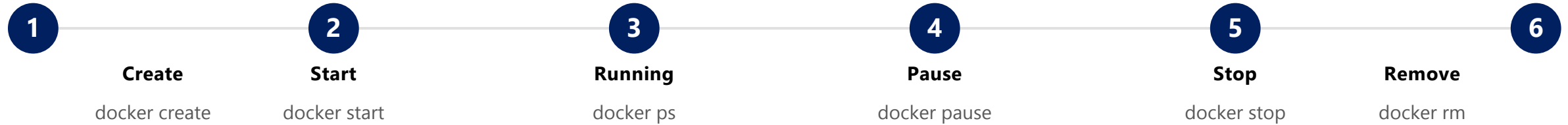
- **1-2. Fill blanks** — JDK/JRE image tags, copy paths, USER, and HEALTHCHECK using course conventions.
- **3. Peer check** — mark any blank you're unsure about for Lab 41 confirmation.
- **4. Security scrub** — confirm no secret ARG/ENV slipped in.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-dockerfile-todos.md.

RUNNING CONTAINERS

A container is a runnable instance of an image. docker run starts a lightweight, isolated process with options for resources, network, storage, and environment.

CONTAINER LIFECYCLE



COMMON docker run OPTIONS

- **-d** — run in detached mode (background).
- **-p host:container** — publish a container port to the host.
- **-e KEY=VALUE** — set an environment variable.
- **-v vol:/path** — mount a volume for persistent data.
- **--name** — assign a readable container name.
- **--rm** — remove the container automatically on exit.

Example: run the CRM container

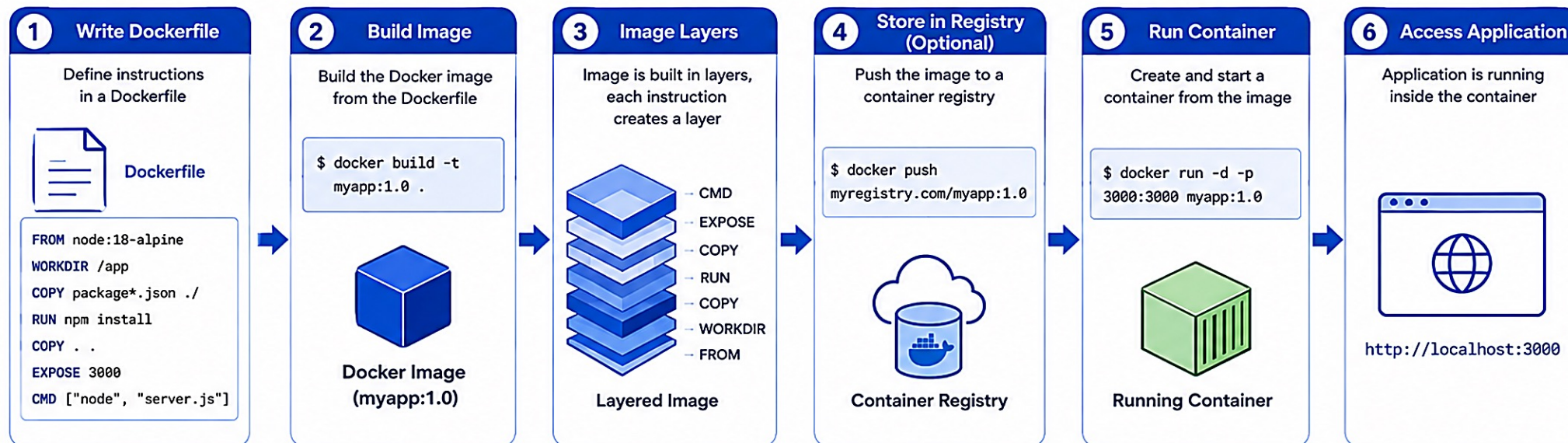
```
docker run -d --name crm-api \
  -p 8080:8080 --memory=512m \
  --env-file .env.local \
  crm-api:lab41

curl -fsS http://localhost:8080/\
  actuator/health/readiness
```

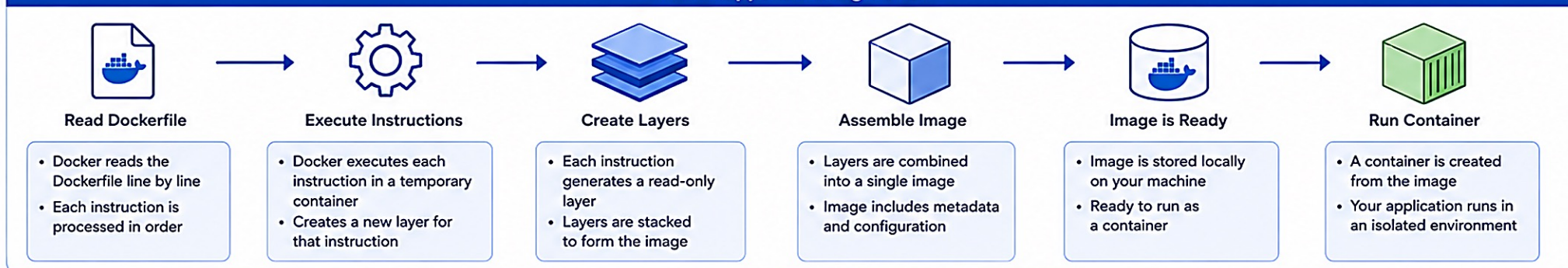
KEY TAKEAWAY Use docker run with the right options to run containers securely, efficiently, and predictably in any environment.

Building Containers

From Dockerfile to Running Container



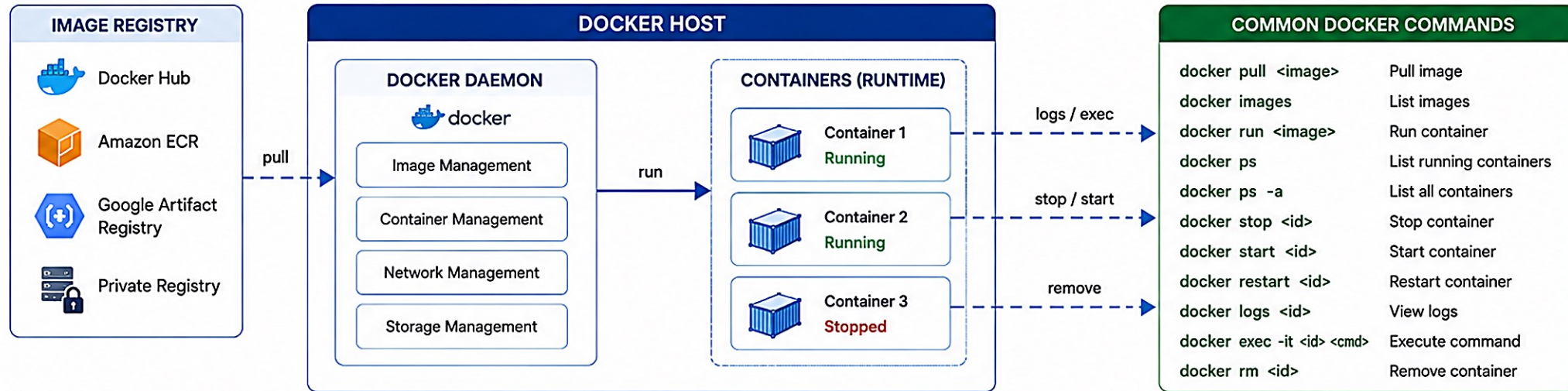
What Happens During Build?



Best Practices



RUNNING CONTAINERS



MANAGING CONTAINERS

Docker provides a rich set of commands to manage the full lifecycle of containers -- list, inspect, monitor, limit, and remove them safely.

Command	Description
docker ps / ps -a	List running / all containers.
docker logs -f <name>	Follow live container logs.
docker stats <name>	Live CPU/memory/network usage.
docker inspect <name>	Detailed container info (JSON).
docker exec -it <name> sh	Open a shell inside the container.
docker rm / rm -f	Remove a stopped / running container.

RESOURCE LIMITS & RESTART POLICIES

```
docker run -d --name app \  
  --cpus="1.0" --memory="512m" \  
  --restart unless-stopped myapp:1.0
```

- **no (default)** — never restart automatically.
- **on-failure[:n]** — restart if the container exits non-zero.
- **unless-stopped** — always restart unless manually stopped.

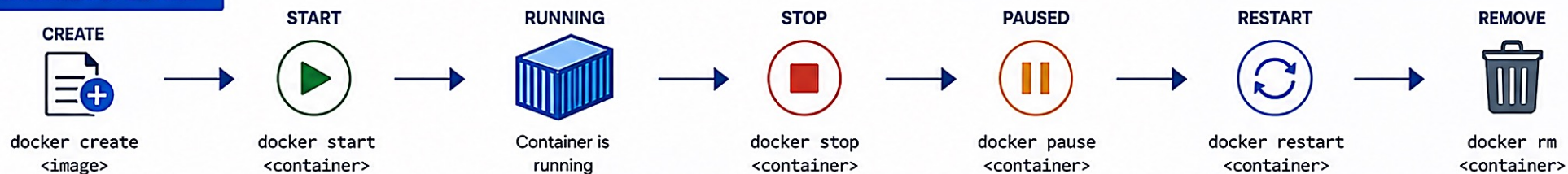
KEEP YOUR ENVIRONMENT CLEAN

- `docker container prune -- remove stopped containers.`
- `docker image prune -a / docker volume prune / docker network prune -- remove unused images, volumes, networks.`

KEY TAKEAWAY A stopped container can be started again. A removed container cannot be recovered -- capture logs and evidence before removing anything.

MANAGING CONTAINERS

CONTAINER LIFECYCLE



LIST & INSPECT

- List all containers
`docker ps -a`
- List running containers
`docker ps`
- Inspect container details
`docker inspect <container>`

MONITOR & LOGS

- View resource usage
`docker stats`
- View logs
`docker logs <container>`
- Follow logs in real-time
`docker logs -f <container>`

EXECUTE COMMANDS

- Open shell in container
`docker exec -it <container> /bin/sh`
- Run command in container
`docker exec <container> <cmd>`
- Run as specific user
`docker exec -u <user> <container> <cmd>`

CONFIGURE & UPDATE

- Connect to network
`docker network connect <net> <container>`
- Update container
`docker update <container> --cpus 1 --memory 512m`
- Rename container
`docker rename <old_name> <new_name>`

RESOURCE MANAGEMENT

- CPU**
 - Limit CPU
`--cpus 1.5`
 - CPU shares
`--cpu-shares 512`
- MEMORY**
 - Limit memory
`--memory 512m`
 - Memory swap
`--memory-swap 1g`
- STORAGE**
 - Mount volume
`-v /data:/app/data`
 - Read-only mount
`-v /data:/app:ro`

NETWORKING

- List networks
`docker network ls`
- Inspect network
`docker network inspect <net>`
- Common Network Commands
 - Create network
`docker network create <net>`
 - Connect container
`docker network connect <net> <container>`
 - Disconnect container
`docker network disconnect <net> <container>`
 - Remove network
`docker network rm <net>`
 - Prune unused networks
`docker network prune`

CLEANUP

- Remove stopped containers
`docker container prune`
- Remove unused images
`docker image prune -a`
- Remove unused volumes
`docker volume prune`
- Remove unused networks
`docker network prune`
- System cleanup
`docker system prune -a`

BEST PRACTICES



Run as non-root user for better security



Limit resources to prevent overuse



Keep containers stateless



Centralize logs and monitoring



Regular cleanup to free space and resources

TRY IT YOURSELF — EXERCISE 3: HEALTH & RESOURCE CHECKLIST

Reinforces: readiness semantics, resource limits, and graceful shutdown from Running and Managing Containers, applied to crm-api.

GOAL: `notes/lab41-health-resources.md`

Step	What to Document
1 -- Health	Name the Actuator readiness path (e.g. /actuator/health/readiness) and what "ready" means for callers.
2 -- Check the reference	Readiness must fail closed if the database is down -- callers should never see a half-ready CRM.
3 -- Resources	Write placeholder memory/CPU limits for a local docker run (lab defaults are fine).
4 -- Graceful stop	One sentence on the SIGTERM / graceful-shutdown expectation for in-flight lab-request-001 calls.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: `exercises/exercise-03-health-resources.md`.

CONTAINER NETWORKING

Docker provides powerful networking features so containers can communicate with each other, with the host, and with external networks.

Driver	Description	Use Case
bridge	Default driver; creates a private network on the host.	Standalone containers that need to communicate.
host	Removes network isolation; uses the host's network directly.	High-performance, low-latency applications.
none	Disables all networking for the container.	Containers that need no network access.
overlay	Used for multi-host container communication.	Clustered services across multiple nodes.

CONTAINER-TO-CONTAINER COMMUNICATION

- **Same network** — containers reach each other by container name (e.g. http://crm-postgres:5432).
- **Different networks** — do NOT communicate by default.
- **Fix** — docker network connect <network> <container>, or run both on one user-defined network.

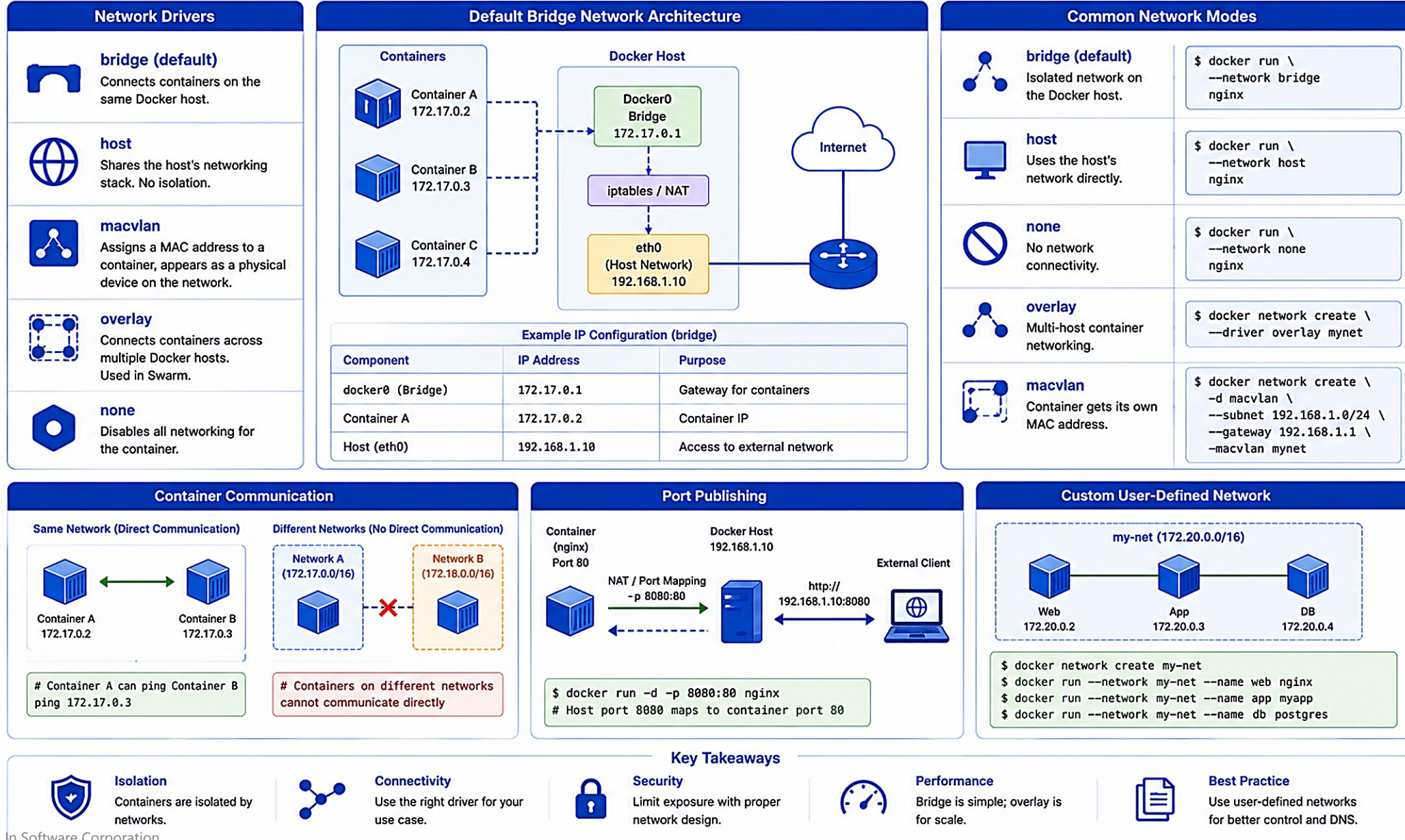
Compose network DNS for crm-postgres

```
docker network create crm-net
docker run -d --name crm-postgres \
  --network crm-net postgres:16
docker run -d --name crm-api \
  --network crm-net \
  -e CRM_DB_URL='jdbc:postgresql://\
    crm-postgres:5432/crm' crm-api:lab41
```

KEY TAKEAWAY Docker networking makes it easy to connect containers securely. localhost inside a container is the container itself -- use the service name or user-defined network DNS instead.

Container Networking

Connect containers to each other and to the outside world



ENVIRONMENT VARIABLES

Environment variables store configuration outside the image, making containers flexible, secure, and portable across environments.

SOURCES OF ENVIRONMENT VARIABLES

- **Dockerfile ENV** — sets a default value baked into the image.
- **docker run -e / --env-file** — injects values at runtime.
- **Docker Compose environment / env_file** — same idea, declared in YAML.
- **Secrets store** — for sensitive values (never plain ENV).

PRECEDENCE (HIGH TO LOW)

- 1. docker run -e / --env-file
- 2. Compose environment / env_file
- 3. Dockerfile ENV
- 4. Default value in application code

Passing env vars with docker run

```
docker run -d --name crm-api \
  -e SPRING_PROFILES_ACTIVE=docker \
  -e CRM_DB_URL=jdbc:postgresql://... \
  -p 8080:8080 crm-api:lab41

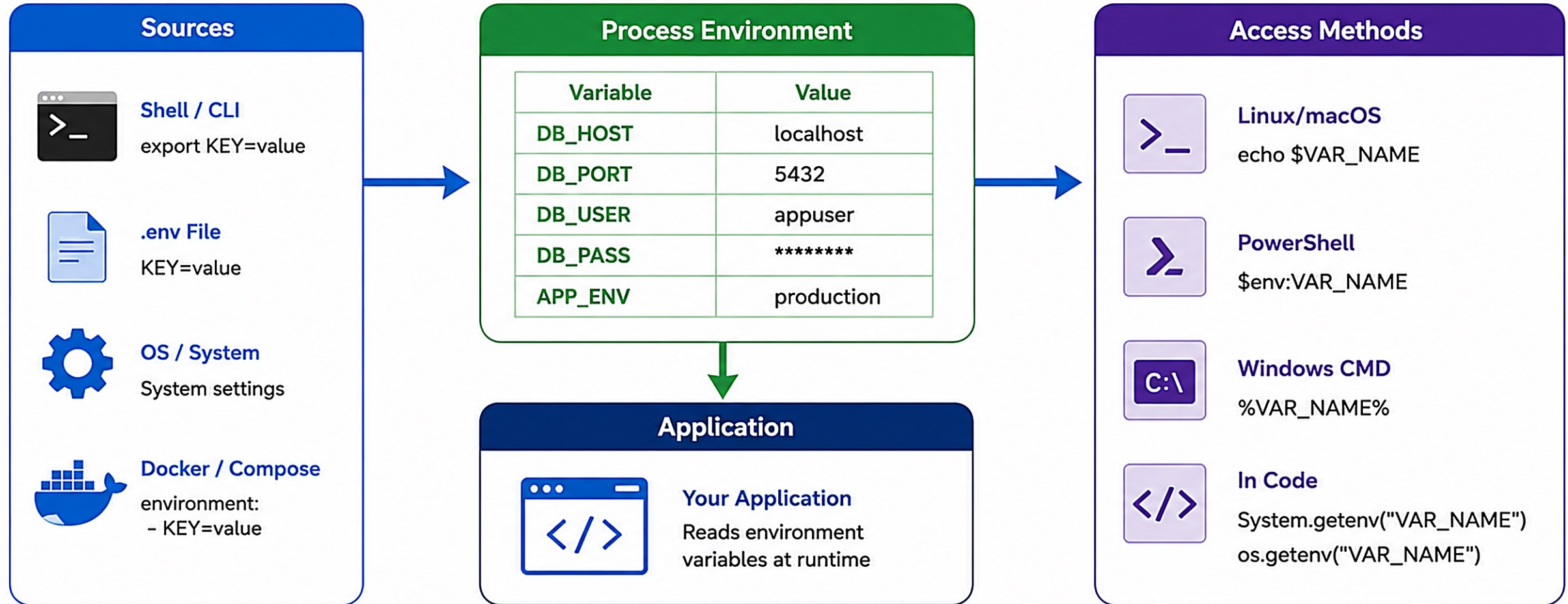
docker run -d --env-file .env.local \
  -p 8080:8080 crm-api:lab41
```

SECURITY

- Never hard-code passwords or API keys in a Dockerfile or Compose file.
- Never COPY a .env file into the image -- inject it at run time only.

KEY TAKEAWAY Environment variables make containers configurable and secure. Use them to adapt crm-api across environments without ever rebuilding the image.

Environment Variables



Benefits



Keep sensitive data out of code



Easy to change per environment



Supports 12-Factor App principles



Perfect for containers and cloud deployments



Improves security and portability

TRY IT YOURSELF — EXERCISE 4: DOCKERIGNORE & ENV PLAN

Reinforces: build-context hygiene and runtime configuration injection, applied together before Lab 41's real Dockerfile.

GOAL: notes/lab41-dockerignore-env.md

.dockerignore candidates

```
target/  
.git/  
.idea/  
.vscode/  
.env  
.env.*  
!.env.example  
*.log  
notes/screenshots/
```

- **1. Ignore list** — draft .dockerignore candidates: .git, target/, .env, notes/, IDE folders.
- **2. Check reference** — runtime config via env -- never ENV CRM_DB_PASSWORD=... in the Dockerfile.
- **3. .env.example** — list keys only, no values: SPRING_DATASOURCE_URL, username, password placeholder.
- **4. Evidence path** — note where Lab 41 stores inspect evidence: notes/screenshots/lab-41/.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-dockerignore-env.md.

DOCKER REGISTRY CONCEPTS

A Docker Registry is a centralized storage and distribution system for images -- store, manage, and share them securely across teams.

Type	Example
Docker Hub	Public default registry (docker.io).
Private Registry	Hosted on your own infrastructure.
Cloud Provider	AWS ECR, Azure ACR, Google GAR.
Hybrid	Combines public and private registries.

BEST PRACTICES

- Use meaningful tags (e.g., 1.0.0), not just :latest.
- Use private registries for sensitive images; enable authentication.
- Scan images for vulnerabilities before pushing.
- Use role-based access control and least-privilege access.
- Clean up old, unused images regularly.

IMAGE NAMING CONVENTION

<registry>/<namespace>/<repository>:<tag>

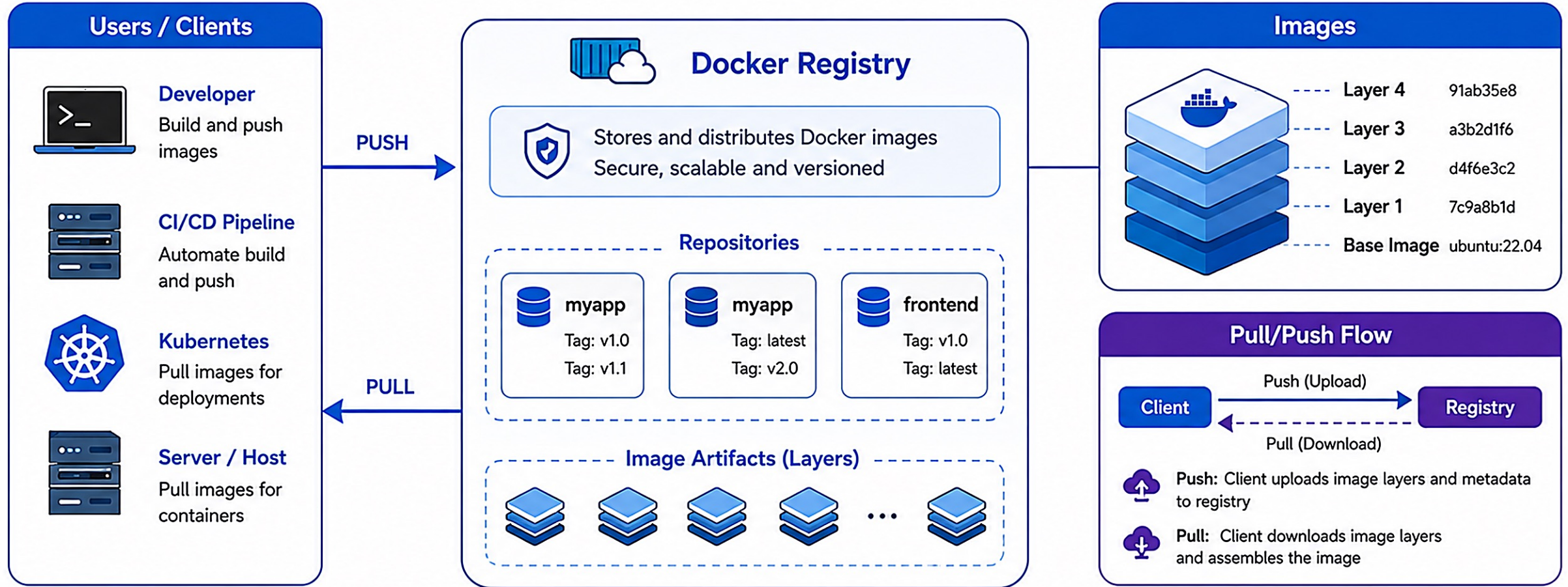
Example: registry.company.com/team/crm-api:1.0.0-a1b2c3d

AUTHENTICATION

docker login registry.company.com -- never store registry credentials in Git; use a credential helper or CI secret.

KEY TAKEAWAY Docker registries are the backbone of container image distribution. Always pin images by specific tag or digest instead of latest for reproducible deployments.

Docker Registry Concepts



Key Concepts



Registry

Central place to store and distribute images



Repository

Namespace for organizing related images



Tag

A reference to a specific image version



Image

A read-only template made of layers



Authentication

Secures access to private images

IMAGE PUBLISHING WORKFLOW

Publishing is the process of building an image locally, tagging it, authenticating, pushing it to a registry, and making it available to pull and run.



Tagging for Lab 42 (version + Git SHA)

```
GIT_SHA=$(git rev-parse --short HEAD)
docker tag crm-api:lab41 \
  crm-api:1.0.0-${GIT_SHA}

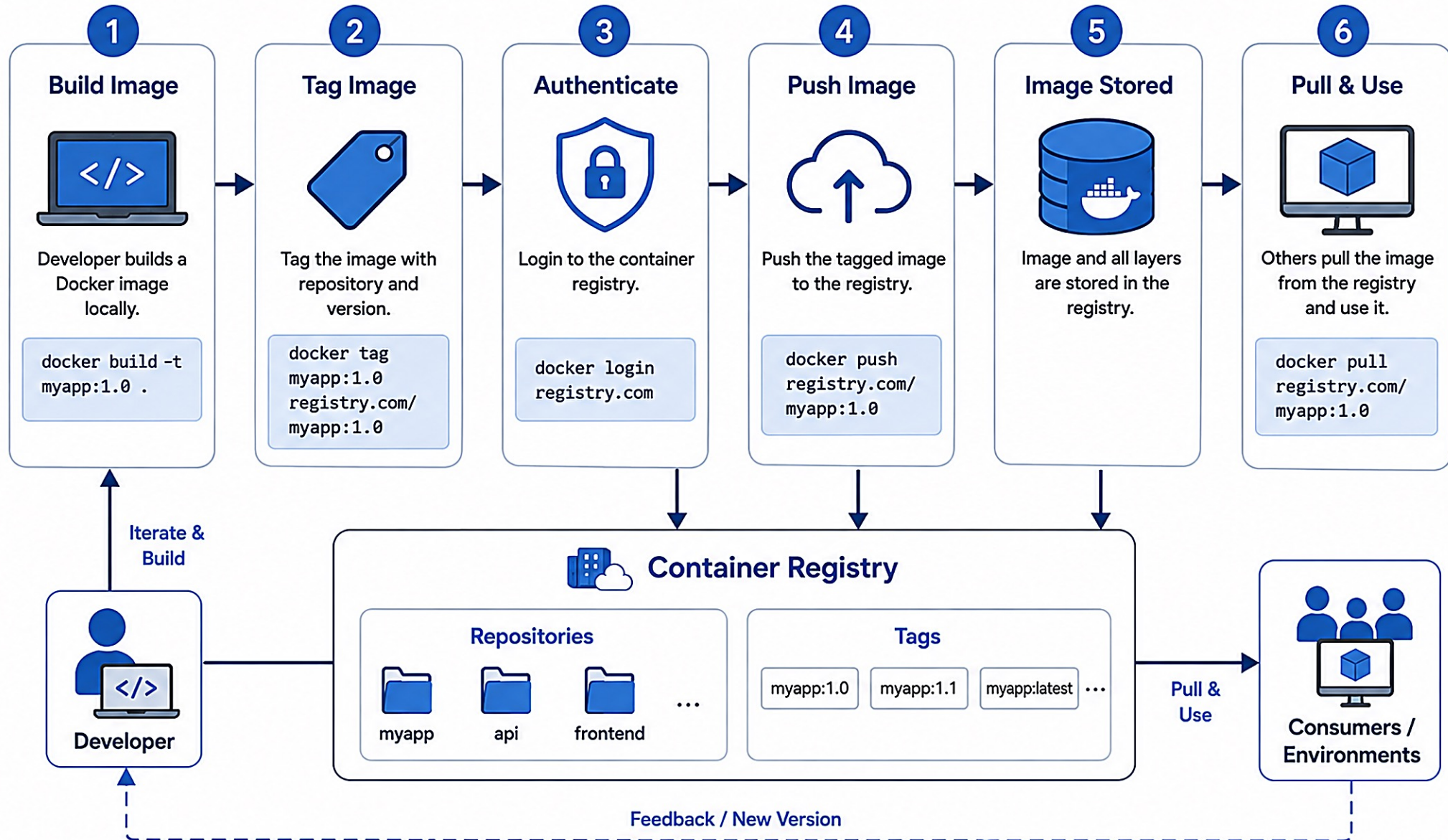
# docker login registry.example.com
# docker push registry.example.com/\
#   training/crm-api:1.0.0-${GIT_SHA}
```

IMAGE TAGGING BEST PRACTICES

- **SemVer (1.0.0)** — stable release tag.
- **Git SHA (a1b2c3d)** — traceability back to source.
- **Digest (sha256:...)** — immutable identity for promotion.
- **Avoid** — shipping :latest-only images to production.

KEY TAKEAWAY Pin images by digest, not just tag, for production promotion. Lab 42 deploys crm-api by digest, not by the mutable :latest tag.

Image Publishing Workflow



TRY IT YOURSELF — EXERCISE 5: DIGEST VS LATEST

Reinforces: why :latest is insufficient for Lab 42's k3s deployment, and what a container-runbook.md must document.

GOAL: notes/lab41-digest-discipline.md

- **1. Define** — in two sentences, define an image digest vs. a mutable tag.
- **2. Check the reference** — Labs 42/44 promote by digest; :latest can drift between staging and prod.
- **3. CRM example** — write an example tag scheme, crm-api:lab41, plus a digest note placeholder (sha256:...).
- **4. Runbook heading** — add docs/container-runbook.md headings: build, inspect user, run, stop, digest capture.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-digest-discipline.md.

TRY IT YOURSELF — EXERCISE 6: PLAN CONTAINER SMOKE

The final pre-lab checkpoint: plan a post-start smoke test using synthetic customers only, before opening the real Lab 41 guide.

GOAL: notes/lab41-smoke-plan.md



- **1. Steps** — order the happy path: health ready -> create/get CUS-1001 -> correlation header lab-request-001 -> stop container.
- **2. Check the reference** — evidence is screenshots/logs under lab-41 notes, never production dumps.
- **3. Failure case** — plan one negative test: a wrong DB URL should fail readiness.
- **4. Scope line** — state this is a plan only -- the full docker build/run is Lab 41 itself.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-smoke-plan.md -- then open the Lab 41 OS guide.

LAB OVERVIEW — CONTAINERIZE THE SPRING BOOT CRM

Package crm-api as a small, reproducible, non-root image: multi-stage build, hardened JRE runtime, health checks, resource limits, and a container runbook.



Duration

~45 min timed path (starter) in class; full path 3-4 hours for homework depth.



Prerequisites

Java 21 + Maven wrapper green, Docker Engine, Actuator health enabled, Labs 39-40 complete.



Scenario

NorthStar CRM crm-api. Fixtures: CUS-1001 Amina Khan (ACTIVE), CUS-1002 Ravi Singh (PROSPECT).

LEARNING OBJECTIVES

- Explain image layers and build-context hygiene; create a multi-stage Maven -> JRE Dockerfile for Java 21.
- Run Spring Boot as a fixed non-root UID; inject profile, JDBC, and broker settings at runtime.
- Add a container HEALTHCHECK aligned with readiness; inspect image size, user, entrypoint, and digest.
- Troubleshoot startup, health, and graceful shutdown; document registry auth, tags, and digest pinning.

WHAT SUCCESS LOOKS LIKE

Under ~/java-bootcamp/examples/lab41-crm/, the image runs as UID 10001, readiness returns success, CRM fixtures work with correlation lab-request-001, and the runbook lists exact build/run/stop commands with digest evidence.

KEY TAKEAWAY No production promotion of images that run as root, embed .env, or lack readiness signals -- you own that packaging gate for the API that serves Amina and Ravi.

LAB TASKS & DELIVERABLES

lab41-crm -- implementation steps, expected deliverables, and the 100-mark evaluation rubric.

Step	What to Do
1. Prepare	Confirm the built JAR; create .dockerignore excluding target/, .git/, .env*.
2. Multi-Stage Dockerfile	Maven+JDK21 build stage -> eclipse-temurin:21-jre runtime stage.
3. Harden Runtime	USER 10001; no secrets in ENV/ARG; no package-manager leftovers.
4. Build & Inspect	docker build --pull; inspect Id, Size, Config.User, RepoDigests.
5. Run with Config	--env-file .env.local, --memory=512m, publish port 8080.
6. Verify Health & CRM	curl actuator/health/readiness; create/get CUS-1001 with correlation header.
7. Shutdown & Failure	docker stop --time 20; invalid JDBC URL -> bounded readiness failure.
8. Document Registry Flow	docs/container-runbook.md: tagging, digest pinning, cleanup.

Build, inspect, verify

```
docker build --pull -t crm-api:lab41 .
docker image inspect crm-api:lab41 \
  --format '{{.Id}} {{.Size}} \
  {{json .Config.User}}' \
  curl -fsS http://localhost:8080/\
    actuator/health/readiness
docker stop --time 20 crm-api
```

EXPECTED DELIVERABLES

- Dockerfile (multi-stage, non-root, health) + .dockerignore + .env.example.
- Image build evidence (id/size/user) + digest notes.
- Readiness + CRM smoke evidence (CUS-1001); graceful stop + dependency-failure evidence.
- docs/container-runbook.md; no secrets in Git or image layers.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Implementation 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Root runtime loses security marks. Password ENV in the Dockerfile is an honor violation. A runbook without inspect evidence loses documentation marks.

MODULE 41 SUMMARY

In this module, you learned how to containerize a Spring Boot application using Docker -- images, containers, networking, environment variables, and registries.

KEY CONCEPTS COVERED



Images & Containers

Immutable, layered images; ephemeral, isolated running containers.



Multi-Stage Dockerfiles

Build stage (Maven+JDK) separate from a lean, non-root runtime stage.



Non-Root & Health

Fixed UID 10001, HEALTHCHECK aligned with actuator readiness.



Networking & Env Config

Bridge networks, container-name DNS, and runtime-injected environment variables.



Registries & Publishing

Tagging, digest pinning, and the build -> tag -> push -> pull -> run workflow.



Hands-on Lab

Built lab41-crm: multi-stage Dockerfile, non-root user, health, and runbook.

MODULE FLOW RECAP

1

Dockerfile

2

Build Image

3

Run Container

4

Push to Registry

5

Lab: lab41-crm

KEY TAKEAWAY Docker standardizes how applications are built, shipped, and run. Containers are portable, lightweight, and isolated -- and best practices ensure secure, production-ready images.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of Docker images, the build/run commands, and port mapping. Choose the best answer for each question.

1. What is a Docker image?

- A. A running instance of a container
- B. A lightweight, standalone runnable package**
- C. A virtual machine snapshot
- D. A Dockerfile template

3. Which command is used to run a container from an image?

- A. docker start
- B. docker run**
- C. docker execute
- D. docker launch

2. Which command is used to build a Docker image?

- A. docker create
- B. docker build**
- C. docker make
- D. docker compile

4. Which flag maps a host port to a container port in docker run?

- A. -p**
- B. -P
- C. --port
- D. --map

KEY TAKEAWAY A Docker image is a standalone, self-contained package; docker build creates it; docker run starts a container from it; -p maps a host port to a container port.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on environment variables, listing containers, viewing logs, and the purpose of a Docker volume.

5. Environment variables in Docker are typically set using which option?

- A. -e or --env**
- B. -v
- C. --mount
- D. -c

7. Which Docker command is used to view the logs of a container?

- A. docker view <container>
- B. docker inspect <container>
- C. docker logs <container>**
- D. docker output <container>

6. Which Docker command shows running containers?

- A. docker images
- B. docker ps**
- C. docker list
- D. docker status

8. What is the purpose of a Docker volume?

- A. To expose ports
- B. To persist data outside the container**
- C. To build images faster
- D. To compress images

KEY TAKEAWAY -e/--env sets environment variables; docker ps lists running containers; docker logs views them; a volume persists data outside the container's ephemeral writable layer.

KNOWLEDGE CHECK (3 OF 3)

The final four questions: pushing and pulling images, which file defines a Docker image, and a key advantage of containers over VMs.

9. Which command is used to push an image to a registry?

- A. docker upload
- B. docker push**
- C. docker publish
- D. docker send

11. Which command is used to pull an image from a registry?

- A. docker fetch
- B. docker pull**
- C. docker download
- D. docker retrieve

10. Which file is used to define a Docker image?

- A. docker-compose.yml
- B. Dockerfile**
- C. docker.json
- D. build.yaml

12. What is a key advantage of containers over traditional VMs?

- A. They require a hypervisor
- B. They are lightweight and start faster**
- C. They provide a full OS for each app
- D. They are more expensive

KEY TAKEAWAY docker push publishes; a Dockerfile defines the image; docker pull retrieves it; containers are lightweight and start in seconds because they share the host OS kernel, unlike VMs.

MODULE 41 COMPLETE

Containerization with Docker

You can now build Docker images, write multi-stage Dockerfiles, run and manage non-root containers, configure networking and environment variables, and publish images to a registry.